MVP Project Plan

QSC Take Home Round 2.pdf
PDF

I have to work on this project for an interview and present it. Can you read the guidelines & instructions? I want to start off by making a plan of the small subtasks I need to accomplish as well as the extra subtasks I can go back and add once I have all the basic required specifications implemented.

Yes — I read through the guidelines and instructions in your PDF. The project is an MVP Word on the Web add-in called **Citation Assist**. The core workflow is: select text in Word, send it to the backend, receive a citation object, highlight the text, show the citation in the task pane, and let the user click the citation to jump back to the highlighted text. The MVP is explicitly scoped to Word on the Web, session-only mapping, basic loading/error states, and no persistence or bibliography export. QSC Take Home Round 2

QSC Take Home Round 2
Here's a clean project plan you can work from.

1) First, define the MVP you are actually building

Your minimum acceptable demo should prove these requirements:

- get current selection from Word
- call `POST /analyze`
- receive `citation_text` and `source_id`
- highlight the selected text
- show citation entries in the task pane
- keep an in-memory mapping during the session
- handle no-selection and API-error cases
- bonus: clicking a citation re-selects the linked text QSC Take Home Round 2

That means your project should be framed around one sentence in the interview:

"I prioritized the end-to-end citation workflow first, then added polish only after the full loop worked reliably."

---

2) Required subtasks: build these first

Phase A — Setup and environment

1. Read the PRD and convert it into a checklist.
2. Scaffold the Word add-in with **React + TypeScript** using the Office Add-ins Development Kit.

3. Confirm local preview works and the task pane opens in Word.
4. Verify the backend is reachable with `GET /health`.
5. Test `GET /document` and `POST /analyze` separately before wiring them into the UI. The PDF explicitly gives you these endpoints and expected usage. QSC Take Home Round 2 QSC Take Home Round 2

Phase B — Core Word integration

6. Implement a helper to read the current text selection with Office.js.
7. Handle the “no text selected” case immediately in the UI.
8. Add a small state model for:
    - selected text
    - loading
    - error
    - citation list
    - mapping between citation entry and selected text/range metadata

The technical guide also points you to `taskpane.tsx` as the main UI location and shows that Word operations should be wrapped in `Word.run()` with `context.sync()`. QSC Take Home Round 2

Phase C — API integration

9. Create a typed API client for `/analyze`.
10. Send JSON payload with:
    - `text`
    - `document_id`
    - `user_id`
11. Parse response fields:
    - `source_id`
    - `citation_text`
    - optional `confidence`
    - optional `url`
12. Handle malformed/empty response safely. QSC Take Home Round 2 QSC Take Home Round 2

Phase D — Citation workflow UI

13. Add an **Analyze** button.
14. Add loading feedback while the request is in flight.
15. After success:
    - highlight the selected text

- add citation entry to a scrollable list in the task pane

16. Make the task pane readable and demo-friendly:

- clear status message
- citation cards/list items
- latest citation visible at top or bottom

17. Make sure repeated use works more than once in the same session.

QSC Take Home Round 2 QSC Take Home Round 2

Phase E — In-memory linking

18. Store a session-only mapping between each citation entry and the associated text/range reference or fallback context.
19. Implement click-on-citation behavior to re-select the associated text.
20. If range objects are unstable, use a fallback strategy like storing the text/context and re-querying, which the PRD itself mentions as a mitigation.

QSC Take Home Round 2 QSC Take Home Round 2

Phase F — Failure states and reliability

21. No selection error state.
22. API failure state with retry.
23. Empty or low-confidence response message.
24. Timeout or slow-response handling so the UI stays responsive.
25. Confirm you are only sending selected text, not scanning the full document.

QSC Take Home Round 2

QSC Take Home Round 2

Phase G — Demo readiness

26. Prepare a short demo document with 2–3 sample passages.
27. Rehearse the happy path live:

- select text
- click analyze
- see citation
- click citation
- jump back to text

28. Rehearse one failure case too.
29. Clean up naming, comments, and README so your design choices are easy to explain.

---

3) Extra subtasks to add only after the core works

These are good “stretch” items because they align with the PRD but are not necessary for the MVP.

High-value polish

- Retry button for transient API failures
- Better loading states and disabled button behavior
- More distinctive highlight color
- Better citation list UX: timestamps, latest-first, selected-state
- Empty state for “no citations yet”

Bonus features from the document

- Clicking a citation to re-select text if you haven’t done it yet
- Insert a comment where the citation is included
- Remove citation / remove highlight action
- Inline citation marker experiment
- Show optional confidence score in UI, if tasteful and not distracting QSC Take Home Round 2 QSC Take Home Round 2

QSC Take Home Round 2

Engineering polish

- Small TypeScript interfaces for API requests/responses
- A separate service file for Word operations
- A separate service file for backend calls
- Component split: **`AnalyzeButton`**, **`CitationList`**, **`StatusBanner`**
- Better error logging for debugging
- Health-check badge in the pane

Nice interview extras

- Explain why you intentionally did not implement persistence, auth, bibliography export, or full-document scanning because they are explicitly out of scope for MVP. That shows product judgment.

QSC Take Home Round 2

---

4) Suggested build order

This is the order I’d use so you get to a working demo fast:

1. Scaffold add-in and run locally
2. Confirm task pane renders
3. Verify backend **`/health`** and **`/analyze`** manually
4. Read selected text from Word
5. Add Analyze button

6. Call backend and log response
7. Render returned citation in pane
8. Highlight selected text
9. Store in-memory citation mapping
10. Click citation to re-select text
11. Add no-selection and API-error states
12. Add polish and stretch features

That order matches the required user flow and minimizes risk because you validate the core integration before spending time on UI polish.

---

5) A practical task board you can copy into Notion/Trello

Must-have

- [ ] Scaffold Word add-in with React + TypeScript
- [ ] Run add-in locally in Word
- [ ] Test backend health endpoint
- [ ] Test backend analyze endpoint
- [ ] Read current selected text from Word
- [ ] Show error if no selection
- [ ] Add Analyze button
- [ ] Send selected text to backend
- [ ] Parse citation response
- [ ] Highlight selected text in document
- [ ] Show citation list in task pane
- [ ] Store session mapping
- [ ] Re-select text when citation is clicked
- [ ] Handle API failure
- [ ] Handle empty/invalid response
- [ ] Rehearse demo flow

Nice-to-have

- [ ] Retry button
- [ ] Health status indicator
- [ ] Confidence display
- [ ] Comment insertion

- [ ] Remove citation / remove highlight
- [ ] Cleaner component structure
- [ ] Better visual styling
- [ ] README with design decisions and tradeoffs

---

6) What to say in the presentation

Frame your work around product choices, not just code.

You can say:

- The MVP target was a **low-friction citation workflow** for Word on the Web.
  QSC Take Home Round 2
- I prioritized the required end-to-end flow before stretch features.
- I kept citation mappings in memory because the PRD explicitly prefers session-only storage to avoid cross-session range instability.
  QSC Take Home Round 2
- I used highlighting instead of inline citation insertion because the design favors minimal document mutation and fewer formatting conflicts.
  QSC Take Home Round 2
- I built error handling around the listed failure cases: no selection, API failure, and empty/low-confidence response.
  QSC Take Home Round 2

Also, the PDF includes setup guidance and a sample repo for a Word add-in using the Office Add-ins Development Kit, React, and TypeScript, which is useful for your starting point.
QSC Take Home Round 2

QSC Take Home Round 2
The uploaded file is here:
QSC Take Home Round 2
Next I can turn this into a **day-by-day implementation schedule** or a **GitHub issues style checklist**.

Sources

> I have to complete this in 24 hours. Can you make a GitHub style checklist?

Yep — here's a **GitHub-style checklist** you can paste directly into an issue or README.

Citation Assist — 24 Hour Build Checklist

0. Project understanding

- [ ] Read the MVP scope and write down the exact required flow: select text → analyze → receive citation → highlight text → show citation in task pane → click citation to jump back to text.

  QSC Take Home Round 2
- [ ] Note the required failure cases: no selection, API failure, empty/low-confidence response.

  OSC Take Home Round 2

- [ ] Note the out-of-scope features so I do not waste time on them: auth, persistence, full-document scanning, bibliography export, collaboration, multi-style citation switching.

QSC Take Home Round 2

1. Environment + scaffolding

- [ ] Install prerequisites: VS Code, Node.js LTS, Microsoft 365 account, Office Add-ins Development Kit.

QSC Take Home Round 2

- [ ] Scaffold a new **Word Add-in** with **React** and **TypeScript**.

QSC Take Home Round 2

- [ ] Open local preview and confirm the task pane renders correctly.
- [ ] Keep the sample project handy as a reference.

QSC Take Home Round 2

- [ ] Confirm the dev server is running and fix any blank task pane or port issues if needed.

QSC Take Home Round 2

2. Backend connectivity

- [ ] Test `GET /health` to confirm the backend is reachable.

QSC Take Home Round 2

- [ ] Test `GET /document` to inspect the source material used by the backend.

QSC Take Home Round 2

- [ ] Test `POST /analyze` manually with sample text before connecting it to the UI.

QSC Take Home Round 2

- [ ] Confirm expected response fields: `source_id`, `citation_text`, optional `confidence`, optional `url`.

QSC Take Home Round 2

3. Core Office.js integration

- [ ] Add logic to read the current selected text in Word using Office.js.

QSC Take Home Round 2

- [ ] Prevent empty submissions when no text is selected.

QSC Take Home Round 2

- [ ] Show inline message: “Please select text before analyzing.”

QSC Take Home Round 2

- [ ] Add a clear **Analyze** button in the task pane.
- [ ] Confirm selection reading works repeatedly in the same session.

4. API integration

- [ ] Create a typed API helper for `POST /analyze`.
- [ ] Send required request body with `text`, plus `document_id` and `user_id` if used.

QSC Take Home Round 2

- [ ] Handle loading state during the network request.

QSC Take Home Round 2

- [ ] Handle malformed responses safely.
- [ ] Show inline API error state if the request fails.

QSC Take Home Round 2

- [ ] Add retry behavior for transient failures.

QSC Take Home Round 2 QSC Take Home Round 2

5. Citation rendering workflow

- [ ] After successful analysis, highlight the selected text in the document.

QSC Take Home Round 2

- [ ] Display the returned `citation_text` in a scrollable list in the task pane.

QSC Take Home Round 2

- [ ] Make sure the UI remains responsive while the network request runs.
QSC Take Home Round 2
- [ ] Confirm no unintended formatting or document corruption occurs.
QSC Take Home Round 2
- [ ] Use highlighting rather than inline citation insertion for the main MVP.
QSC Take Home Round 2

6. Session mapping

- [ ] Maintain an in-memory mapping between citation entries and the analyzed text during the session.
QSC Take Home Round 2
- [ ] Do not implement persistence across sessions.
QSC Take Home Round 2
- [ ] If Word range references become unstable, store fallback context text or re-query selection.

QSC Take Home Round 2

7. Click-to-jump behavior

- [ ] Implement citation click behavior to re-select the associated text.
QSC Take Home Round 2
- [ ] Test this multiple times with multiple citation entries.
- [ ] Confirm it works as a bonus feature for the demo.
QSC Take Home Round 2

8. Error states + edge cases

- [ ] No selection state works cleanly.
QSC Take Home Round 2
- [ ] API failure shows message and retry option.
QSC Take Home Round 2
- [ ] Empty or low-confidence response shows: “Unable to generate citation. Try refining selection.”

QSC Take Home Round 2
- [ ] Timeout fallback does not freeze the UI.
QSC Take Home Round 2
- [ ] Ensure only selected text is sent to the backend.
QSC Take Home Round 2

9. Code cleanup

- [ ] Separate Word logic from API logic.
- [ ] Add TypeScript interfaces for request/response objects.
- [ ] Keep components simple and easy to explain in the interview.
- [ ] Remove dead code and placeholder sample UI.
- [ ] Add short comments only where they improve clarity.

10. Demo prep

- [ ] Prepare a sample Word document with 2–3 passages to test.
- [ ] Rehearse the happy path end-to-end.
- [ ] Rehearse one failure path too.
- [ ] Be ready to explain why the MVP is scoped to Word on the Web only.
QSC Take Home Round 2
- [ ] Be ready to explain why citation mapping is session-only.
QSC Take Home Round 2

- [ ] Be ready to explain why highlighting was chosen over inline insertion.
QSC Take Home Round 2

Nice-to-have only after core MVP works

- [ ] Show backend health status in the task pane.
QSC Take Home Round 2
- [ ] Show optional confidence score in UI.
QSC Take Home Round 2
- [ ] Add comment insertion as a bonus feature.
QSC Take Home Round 2
- [ ] Add remove citation / remove highlight action.
QSC Take Home Round 2
- [ ] Improve styling and visual feedback for the citation list.
QSC Take Home Round 2

24-hour recommended execution order

- [ ] Hour 1–2: scaffold add-in + run locally
- [ ] Hour 2–3: test backend endpoints
- [ ] Hour 3–6: get selected text from Word + Analyze button
- [ ] Hour 6–9: connect `/analyze` and render returned citation
- [ ] Hour 9–12: add highlighting in document
- [ ] Hour 12–15: add in-memory citation mapping
- [ ] Hour 15–17: implement click citation → re-select text
- [ ] Hour 17–19: handle no-selection, API failure, empty response
- [ ] Hour 19–21: cleanup and UI polish
- [ ] Hour 21–24: rehearse demo + prepare explanation

I can also turn this into a **prioritized GitHub issue list with labels like `must-have`, `bonus`, and `risk`**.

Sources

can you give me this checklist in raw markdown

Here is the **raw Markdown** so you can paste it directly into a GitHub issue, README, or project board.

```
# Citation Assist — 24 Hour Build Checklist

## 0. Project understanding
- [ ] Read the MVP scope and write down the exact required flow: select text → analyze → receive citat
- [ ] Note the required failure cases: no selection, API failure, empty/low-confidence response
- [ ] Note the out-of-scope features so I do not waste time on them: auth, persistence, full-document

---

## 1. Environment + scaffolding
- [ ] Install prerequisites: VS Code, Node.js LTS, Microsoft 365 account, Office Add-ins Development K
- [ ] Scaffold a new Word Add-in with React and TypeScript
- [ ] Open local preview and confirm the task pane renders correctly
- [ ] Keep the sample project handy as a reference
- [ ] Confirm the dev server is running and fix any blank task pane or port issues if needed

---

## 2. Backend connectivity
- [ ] Test `GET /health` to confirm the backend is reachable
- [ ] Test `GET /document` to inspect the source material used by the backend
- [ ] Test `POST /analyze` manually with sample text before connecting it to the UI
- [ ] Confirm expected response fields: `source_id`, `citation_text`, optional `confidence`, optional

---

## 3. Core Office.js integration
- [ ] Add logic to read the current selected text in Word using Office.js
- [ ] Prevent empty submissions when no text is selected
- [ ] Show inline message: "Please select text before analyzing"
- [ ] Add a clear Analyze button in the task pane
- [ ] Confirm selection reading works repeatedly in the same session

---

## 4. API integration
- [ ] Create a typed API helper for `POST /analyze`
- [ ] Send required request body with `text`, plus `document_id` and `user_id` if used
- [ ] Handle loading state during the network request
- [ ] Handle malformed responses safely
- [ ] Show inline API error state if the request fails
- [ ] Add retry behavior for transient failures

---

## 5. Citation rendering workflow
- [ ] After successful analysis, highlight the selected text in the document
- [ ] Display the returned `citation_text` in a scrollable list in the task pane
- [ ] Make sure the UI remains responsive while the network request runs
- [ ] Confirm no unintended formatting or document corruption occurs
- [ ] Use highlighting rather than inline citation insertion for the main MVP

---

## 6. Session mapping
- [ ] Maintain an in-memory mapping between citation entries and the analyzed text during the session
- [ ] Do not implement persistence across sessions
- [ ] If Word range references become unstable, store fallback context text or re-query selection

---

## 7. Click-to-jump behavior
- [ ] Implement citation click behavior to re-select the associated text
- [ ] Test this multiple times with multiple citation entries
- [ ] Confirm it works as a bonus feature for the demo

---

## 8. Error states + edge cases
- [ ] No selection state works cleanly
- [ ] API failure shows message and retry option
- [ ] Empty or low-confidence response shows: "Unable to generate citation. Try refining selection."
- [ ] Timeout fallback does not freeze the UI
- [ ] Ensure only selected text is sent to the backend
```

```
---

## 9. Code cleanup
- [ ] Separate Word logic from API logic
- [ ] Add TypeScript interfaces for request/response objects
- [ ] Keep components simple and easy to explain in the interview
- [ ] Remove dead code and placeholder sample UI
- [ ] Add short comments only where they improve clarity

---

## 10. Demo prep
- [ ] Prepare a sample Word document with 2–3 passages to test
- [ ] Rehearse the happy path end-to-end
- [ ] Rehearse one failure path too
- [ ] Be ready to explain why the MVP is scoped to Word on the Web only
- [ ] Be ready to explain why citation mapping is session-only
- [ ] Be ready to explain why highlighting was chosen over inline insertion

---

# Nice-to-have only after core MVP works

- [ ] Show backend health status in the task pane
- [ ] Show optional confidence score in UI
- [ ] Add comment insertion as a bonus feature
- [ ] Add remove citation / remove highlight action
- [ ] Improve styling and visual feedback for the citation list

---

# 24-hour recommended execution order

- [ ] Hour 1–2: scaffold add-in + run locally
- [ ] Hour 2–3: test backend endpoints
- [ ] Hour 3–6: get selected text from Word + Analyze button
- [ ] Hour 6–9: connect `/analyze` and render returned citation
- [ ] Hour 9–12: add highlighting in document
- [ ] Hour 12–15: add in-memory citation mapping
- [ ] Hour 15–17: implement click citation → re-select text
- [ ] Hour 17–19: handle no-selection, API failure, empty response
- [ ] Hour 19–21: cleanup and UI polish
- [ ] Hour 21–24: rehearse demo + prepare explanation
```

If you want, I can also give you a **super optimized "minimum-to-pass-the-interview" checklist (about 12 tasks)** so you reach a working demo in ~6–8 hours before polishing.

> I'm happy with this checklist - I've followed the instructions and I have my starting add-in created. Next step is testing the different request methods to ensure correct backend connectivity. Where should this could go (which file - or do I create a new one)

Great — since you already scaffolded the **Word Add-in with React + TypeScript**, the best practice is **not** to test backend requests inside the UI component directly. Instead, create a **separate API service file** and call it from your React component.

This keeps your code organized and also looks better in an interview because it shows **separation of concerns**.

Recommended structure

Your project likely looks roughly like this:

```
src/
  taskpane/
    components/
    taskpane.tsx
  commands/
  index.ts
```

Add a new folder:

```
src/
  services/
    api.ts
```

So the structure becomes:

```
src/
  taskpane/
    taskpane.tsx
  services/
    api.ts
```

Step 1 — Create the API helper file

Create:

```
src/services/api.ts
```

Example implementation:

```
const BASE_URL = "YOUR_BACKEND_URL";

export async function checkHealth() {
  const response = await fetch(`${BASE_URL}/health`);

  if (!response.ok) {
    throw new Error("Health check failed");
  }

  return response.json();
}

export async function getDocument() {
  const response = await fetch(`${BASE_URL}/document`);

  if (!response.ok) {
    throw new Error("Failed to fetch document");
  }

  return response.json();
}

export async function analyzeText(text: string) {
  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      text,
      document_id: "demo_doc",
      user_id: "demo_user",
    }),
  });

  if (!response.ok) {
    throw new Error("Analyze request failed");
  }

  return response.json();
}
```

Step 2 — Call the API from your React component

Inside:

```
src/taskpane/taskpane.tsx
```

Add a simple test button.

Example:

```
import { checkHealth, getDocument, analyzeText } from "../services/api";

async function testBackend() {
  try {
    const health = await checkHealth();
    console.log("Health:", health);

    const doc = await getDocument();
    console.log("Document:", doc);

    const analysis = await analyzeText("Sample test sentence");
    console.log("Analyze result:", analysis);
  } catch (error) {
    console.error(error);
  }
}
```

Then add a button in your JSX:

```
<button onClick={testBackend}>Test Backend</button>
```

Step 3 — Verify in the browser console

Run the add-in:

```
npm start
```

Open the **Word taskpane devtools console** and click:

```
Test Backend
```

You should see logs like:

```
Health: { status: "ok" }
Document: {...}
Analyze result: {...}
```

If this works, **your backend connectivity is confirmed**.

Why this approach is ideal (for the interview)

It demonstrates:

- modular architecture
- separation of UI and API logic
- reusable backend services
- clean React component structure

You can say:

> "I separated backend communication into a service layer so the UI components remain focused on interaction and state management."

Interviewers love hearing that.

Next step after connectivity (important)

Once your backend tests pass, your **very next task** should be:

`getSelectedTextFromWord()`

because the full workflow is:

`Word selection → analyze API → highlight → citation list`

If you want, I can also show you the **cleanest Office.js function to get the selected text**, because that's the part that usually causes people trouble in Word add-ins.

> Ok great, I've fully integrated these methods into my server/api.ts and tested them for successful responses. The next component is adding logic to read the current selected text in Word using Office.js. This can be done using the following API operation:
>
> Get selected text context.document.getSelection()
>
> For now, let's set it up to have the user select some text, then press a button and log the text they printed in the console

Yes — that's the right next step.

For this, keep the Word logic in a separate helper too, just like you did for the API layer. That makes it easier to test and easier to explain.

Where to put it

Create a new file like:

`src/services/word.ts`

Then put the Office.js selection logic there, and call it from your task pane button.

---

1. Create `src/services/word.ts`

```
export async function getSelectedText(): Promise<string> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();
    selection.load("text");

    await context.sync();

    return selection.text.trim();
  });
}
```

This does:

- gets the current Word selection
- loads the selected text
- syncs with the Word document
- returns the trimmed text

---

2. Import it into your task pane component

In your **`taskpane.tsx`**:

```
import React from "react";
import { getSelectedText } from "../services/word";
```

If your file structure is slightly different, adjust the relative path.

---

3. Add a button handler

```
const handleLogSelectedText = async () => {
  try {
    const selectedText = await getSelectedText();

    if (!selectedText) {
      console.log("No text selected");
      return;
    }

    console.log("Selected text:", selectedText);
  } catch (error) {
    console.error("Error getting selected text:", error);
  }
};
```

---

4. Add the button in the JSX

```
<button onClick={handleLogSelectedText}>Log Selected Text</button>
```

---

5. Full simple example in `taskpane.tsx`

Something like this:

```
import React from "react";
import { getSelectedText } from "../services/word";

const Taskpane = () => {
  const handleLogSelectedText = async () => {
    try {
      const selectedText = await getSelectedText();

      if (!selectedText) {
        console.log("No text selected");
        return;
      }

      console.log("Selected text:", selectedText);
    } catch (error) {
      console.error("Error getting selected text:", error);
    }
  };

  return (
    <div>
      <h1>Citation Assist</h1>
      <button onClick={handleLogSelectedText}>Log Selected Text</button>
    </div>
  );
};

export default Taskpane;
```

---

6. How to test it

1. Start your add-in
2. Open the task pane in Word
3. Highlight some text in the document
4. Click **Log Selected Text**
5. Check the task pane console

You should see:

```
Selected text: your highlighted text here
```

If nothing is selected, you should see:

```
No text selected
```

---

7. Why this is a good checkpoint

Once this works, you've confirmed the first half of the full workflow:

```
Word selection -> React button -> Office.js -> selected text
```

Then the next step is just connecting that selected text to your existing **`analyzeText()`** API call.

A nice next move is to change the button handler from:

- get selected text
- log selected text

to:

- get selected text
- send selected text to `/analyze`
- log response

That will give you your first real end-to-end flow.

> ok so I added getSelectedText() in taskpane.ts. I want to create a designated tsk file for the "Analyze" component. Essentially, after the user selects the text, they will click a button that says "Analyze". This .tsx file for the Analyze button will be imported into App.tsx which is the top level tsx for all the components of the UI. If you want to see what the current App.tsx and a sample button currently in for InsertText with its own tsk component file I can share that. For clarity, the InsertText will be modified later

Yes — that's a good structure.

You should make a separate **Analyze button component** in its own `.tsx` file and import it into `App.tsx`. Since you already have an `InsertText` button component pattern in the project, it makes sense to follow the same structure so the codebase stays consistent.

Recommended structure

Something like this:

```
src/
  taskpane/
    components/
      AnalyzeButton.tsx
      InsertText.tsx
    App.tsx
  services/
    api.ts
    word.ts
```

If your generated project uses a slightly different folder layout, that's fine — the main idea is:

- UI button component in `components/`
- Word logic in `services/word.ts`
- backend logic in `services/api.ts`
- page composition in `App.tsx`

What the `AnalyzeButton.tsx` should do right now

For this checkpoint, keep it simple:

1. user selects text in Word
2. clicks **Analyze**
3. component calls `getSelectedText()`

4. logs the selected text to the console

Later, you'll expand it to:
5. call **`analyzeText(selectedText)`**
6. update UI state
7. show citation results

Suggested `AnalyzeButton.tsx`

```
import React from "react";
import { getSelectedText } from "../../services/word";

const AnalyzeButton = () => {
  const handleAnalyze = async () => {
    try {
      const selectedText = await getSelectedText();

      if (!selectedText) {
        console.log("No text selected");
        return;
      }

      console.log("Selected text:", selectedText);
    } catch (error) {
      console.error("Error getting selected text:", error);
    }
  };

  return <button onClick={handleAnalyze}>Analyze</button>;
};

export default AnalyzeButton;
```

Adjust the import path if your folders are arranged differently.

Then in `App.tsx`

```
import React from "react";
import AnalyzeButton from "./components/AnalyzeButton";

const App = () => {
  return (
    <div>
      <h1>Citation Assist</h1>
      <AnalyzeButton />
    </div>
  );
};

export default App;
```

Why this is a good idea

This gives you a clean split:

- **`App.tsx`** = layout / composition
- **`AnalyzeButton.tsx`** = analyze interaction
- **`word.ts`** = Word document operations

That's much cleaner than putting all the Office.js logic directly in `App.tsx`.

One thing I'd recommend now

Even if you are only logging for now, make the component name reflect behavior you'll keep later:

- `AnalyzeButton.tsx` is good
- not something temporary like `LogSelectionButton.tsx`

That way you won't have to rename it later.

Nice next version after this works

After the button successfully logs selected text, update the handler to:

```
const selectedText = await getSelectedText();
const result = await analyzeText(selectedText);
console.log("Analyze result:", result);
```

That will give you your first real end-to-end interaction.

Go ahead and share your current `App.tsx` and the sample `InsertText` component if you want — then I can match the exact structure and imports to your project.

Here is my TextInsertion.tsx:

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, Textarea, tokens, makeStyles } from "@fluentui/react-components";

/* global HTMLTextAreaElement */

interface TextInsertionProps {
  insertText: (text: string) => void;
}

const useStyles = makeStyles({
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  textPromptAndInsertion: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
  },
  textAreaField: {
```

```
    marginLeft: "20px",
    marginTop: "30px",
    marginBottom: "20px",
    marginRight: "20px",
    maxWidth: "50%",
  },
});

const TextInsertion: React.FC<TextInsertionProps> = (props: TextInsertionProps) => {
  const [text, setText] = useState<string>("Some text.");

  const handleTextInsertion = async () => {
    await props.insertText(text);
  };

  const handleTextChange = async (event: React.ChangeEvent<HTMLTextAreaElement>) => {
    setText(event.target.value);
  };

  const styles = useStyles();

  return (
    <div className={styles.textPromptAndInsertion}>
      <Field className={styles.textAreaField} size="large" label="Enter text to be inserted into the
document.">
        <Textarea size="large" value={text} onChange={handleTextChange} />
      </Field>
      <Field className={styles.instructions}>Click the button to insert text.</Field>
      <Button appearance="primary" disabled={false} size="large" onClick={handleTextInsertion}>
        Insert text
      </Button>
    </div>
  );
};

export default TextInsertion;


Here is my app.tsx:

import * as React from "react";
import Header from "./Header";
import HeroList, { HeroListItem } from "./HeroList";
import TextInsertion from "./TextInsertion";
import AnalyzeButton from "./AnalyzeText";
import { makeStyles } from "@fluentui/react-components";
import { Ribbon24Regular, LockOpen24Regular, DesignIdeas24Regular } from "@fluentui/react-
icons";
import { insertText } from "../taskpane";

interface AppProps {
```

```
  title: string;
}

const useStyles = makeStyles({
  root: {
    minHeight: "100vh",
  },
});

const App: React.FC<AppProps> = (props: AppProps) => {
  const styles = useStyles();
  // The list items are static and won't change at runtime,
  // so this should be an ordinary const, not a part of state.
  const listItems: HeroListItem[] = [
    {
      icon: <Ribbon24Regular />,
      primaryText: "Achieve more with Office integration",
    },
    {
      icon: <LockOpen24Regular />,
      primaryText: "Unlock features and functionality",
    },
    {
      icon: <DesignIdeas24Regular />,
      primaryText: "Create and visualize like a pro",
    },
  ];

  return (
    <div className={styles.root}>
      <Header logo="assets/logo-filled.png" title={props.title} message="Welcome" />
      <HeroList message="Discover what this add-in can do for you today!" items={listItems} />
      <TextInsertion insertText={insertText} />
      <AnalyzeButton />
    </div>
  );
};

export default App;
```

Perfect — this is enough. Your structure is clear, and you can mirror `TextInsertion.tsx` for the new analyze component.

A couple things stand out:

- your component files live alongside `App.tsx`, not in a separate `components/` folder
- `App.tsx` is already importing `AnalyzeButton` from `"./AnalyzeText"`
- you're using Fluent UI components, so the Analyze component should follow that style too

What I'd do

Create a new file beside **`TextInsertion.tsx`**, for example:

`AnalyzeText.tsx`

Then build it similarly to **`TextInsertion.tsx`**, but much simpler for now:

- no textarea needed
- just instructions
- one button
- button calls **`getSelectedText()`**
- logs selection to console

Suggested `AnalyzeText.tsx`

```
import * as React from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../services/word";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
});

const AnalyzeButton: React.FC = () => {
  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      const selectedText = await getSelectedText();

      if (!selectedText) {
        console.log("No text selected");
        return;
      }

      console.log("Selected text:", selectedText);
    } catch (error) {
      console.error("Error getting selected text:", error);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>
      <Button appearance="primary" size="large" onClick={handleAnalyze}>
        Analyze
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

Important note about the import path

You wrote that **`getSelectedText()`** is currently in **`taskpane.ts`**.

That will work temporarily, but I would strongly recommend moving it into a service file like:

```
src/taskpane/services/word.ts
```

or whatever matches your structure.

Then import it into **`AnalyzeText.tsx`**.

For example, if your structure is:

```
src/taskpane/components/App.tsx
src/taskpane/components/TextInsertion.tsx
src/taskpane/components/AnalyzeText.tsx
src/taskpane/services/word.ts
```

then this import is correct:

```
import { getSelectedText } from "../services/word";
```

But if **`AnalyzeText.tsx`** is in the same folder as **`App.tsx`** and **`word.ts`** is somewhere else, the relative path may differ.

Based on your current imports, your folder probably looks like this

Something like:

```
taskpane/
  components/
    App.tsx
    Header.tsx
    HeroList.tsx
    TextInsertion.tsx
    AnalyzeText.tsx
  taskpane.ts
```

If that's true, then putting **`word.ts`** here is clean:

```
taskpane/services/word.ts
```

and your import inside **`AnalyzeText.tsx`** becomes:

```
import { getSelectedText } from "../services/word";
```

Your `App.tsx` is already basically ready

Since you already have:

```
import AnalyzeButton from "./AnalyzeText";
```

and:

```
<AnalyzeButton />
```

you probably won't need to change **`App.tsx`** at all, as long as the file name matches.

One small improvement to `App.tsx`

Right now your UI still contains the default sample content:

- **`Header`**
- **`HeroList`**
- **`TextInsertion`**

That's okay for now, but once Analyze works, you'll probably want to simplify the screen so the demo feels focused on your actual feature.

For now though, leave it — just get the Analyze button working first.

Best next move

Do these in order:

1. create **`AnalyzeText.tsx`**
2. move **`getSelectedText()`** into **`services/word.ts`** if you haven't already
3. import it into **`AnalyzeText.tsx`**
4. click Analyze after selecting text in Word
5. confirm selected text prints in console

Once that works, the next update is very small: swap the **`console.log(selectedText)`** for:

```
const result = await analyzeText(selectedText);
console.log("Analyze result:", result);
```

That will give you the first real end-to-end Analyze flow.

If you paste your current **`taskpane.ts`** next, I can show you exactly where **`getSelectedText()`** should be moved and what the correct import paths should be.

> my taskpane.ts only has insertText() and getSelectedText() so a services/word.ts is not needed. Everything is looking good right now except instead of logging "Please select text before analyzing", let's have that as a pop-up that appears to the user without having to click inspect and read the console output

Yes — for this stage, that's totally fine. You do **not** need **`services/word.ts`** yet if **`taskpane.ts`** is only holding **`insertText()`** and **`getSelectedText()`**.

For showing **"Please select text before analyzing"** to the user, the cleanest move is to use a small piece of React state in **`AnalyzeText.tsx`** and render a visible message in the UI instead of logging to the console.

Since you're already using Fluent UI, use a **`Field`** or a simple styled text block for now. That's faster and safer than a browser **`alert()`**, and it looks more polished in an interview.

Update `AnalyzeText.tsx`

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
  },
});

const AnalyzeButton: React.FC = () => {
  const [message, setMessage] = useState<string>("");
  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      const selectedText = await getSelectedText();

      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      setMessage("");
      console.log("Selected text:", selectedText);
    } catch (error) {
      setMessage("Something went wrong while reading the selected text.");
      console.error("Error getting selected text:", error);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze}>
        Analyze
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

What this does

- if no text is selected, the user sees:
  Please select text before analyzing.
- if text is selected, the message disappears

- if Word selection fails, the user sees a visible error message

Why this is better than `alert()`

`alert()` works, but it feels clunky in an add-in. Rendering the message directly in the task pane:

- looks more professional
- is easier to style later
- fits your final MVP better

If you want an actual alert-style popup

You *can* use this:

```
if (!selectedText) {
  window.alert("Please select text before analyzing.");
  return;
}
```

But I would only use that if you specifically want a modal popup. For interview/demo quality, the in-pane message is the better choice.

Your next step after this should be wiring the Analyze button to call your `analyzeText()` request instead of only logging the selected text.

> Okay the select text has been successfully implemented! Next, we need to complete the API helper for POST /analyze **by sending the required request body with** text**, plus** document_id **and** user_id **if used**

Nice — that's the right next milestone.

Since you already tested the backend methods in your `server/api.ts`, this step is about making sure your `POST /analyze` helper is clean, typed, and easy to call from the Analyze button.

What the helper should do

It should:

- accept the selected text
- send it to `/analyze`
- include `document_id` and `user_id` if your backend expects them
- return the parsed JSON response
- throw an error if the request fails

Recommended shape for `server/api.ts`

You can define request/response types first:

```
export interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

export interface AnalyzeResponse {
  source_id: string;
  citation_text: string;
  confidence?: number;
  url?: string;
}
```

Then implement the helper:

```
const BASE_URL = "YOUR_BACKEND_URL";

export async function analyzeText(text: string): Promise<AnalyzeResponse> {
  const requestBody: AnalyzeRequest = {
    text,
    document_id: "demo_doc",
    user_id: "demo_user",
  };

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(requestBody),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  const data: AnalyzeResponse = await response.json();
  return data;
}
```

If `document_id` and `user_id` are not required

You can still keep the type flexible and just omit them:

```
export async function analyzeText(text: string): Promise<AnalyzeResponse> {
  const requestBody: AnalyzeRequest = {
    text,
  };

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(requestBody),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

Then update your `AnalyzeText.tsx`

Right now your button probably:

- gets selected text
- shows message if empty
- logs selected text

Now change it so it also calls `analyzeText(selectedText)`.

Example:

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";
import { analyzeText } from "../server/api";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
  },
});

const AnalyzeButton: React.FC = () => {
  const [message, setMessage] = useState<string>("");
  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      const selectedText = await getSelectedText();

      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      setMessage("");

      const result = await analyzeText(selectedText);
      console.log("Analyze result:", result);
    } catch (error) {
      setMessage("Analyze request failed.");
      console.error("Error analyzing text:", error);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze}>
        Analyze
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

Better version with loading state

This is a stronger version because users shouldn't be able to spam the button while the request is running.

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";
import { analyzeText } from "../server/api";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
  },
});

const AnalyzeButton: React.FC = () => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      const selectedText = await getSelectedText();

      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      setMessage("");
      setIsLoading(true);

      const result = await analyzeText(selectedText);
      console.log("Analyze result:", result);
    } catch (error) {
      setMessage("Analyze request failed.");
      console.error("Error analyzing text:", error);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

What to test right after this

Once you wire it up, test these 4 cases:

1. no text selected → visible message
2. text selected → request succeeds and logs returned JSON
3. backend down → visible error message
4. repeated clicks → button disables while loading

Best next checkpoint

After this works, the next milestone is:

- store the returned analysis result in state
- display `citation_text` in the task pane

That's the moment your app starts feeling like the actual product instead of just a connectivity test.

Paste your current `analyzeText()` helper if you want, and I'll check whether the request body and typing look right.

here is my analyzeText():

```
export async function analyzeText(text: string) {
  const response = await fetch(${BASE_URL}/analyze, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      text,
      document_id: "trustops-handbook-v1",
      user_id: "candidate_1",
    }),
  });

  if (!response.ok) {
    throw new Error(Analyze request failed with status ${response.status});
  }

  return response.json();
}
```

I want to confirm - does this work with document_id and user_id being optional?

Yes — **your function will work even if `document_id` and `user_id` are optional, as long as the backend allows them to be present or absent.**

Right now you are **always sending them**, which is completely safe if the API accepts them.

What your current request body looks like

Your code sends:

```
{
  "text": "selected text here",
  "document_id": "trustops-handbook-v1",
  "user_id": "candidate_1"
}
```

If the API spec says those fields are **optional**, it usually means:

- They **can be included**
- They **can be omitted**

So your implementation is perfectly valid.

When would it be a problem?

Only if the backend strictly expected:

```
{
  "text": "..."
}
```

and **rejected extra fields** — which is extremely rare for APIs.

Most APIs simply ignore extra optional fields.

If you wanted to make them truly optional in your helper

You could write it like this:

```
export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
) {
  const body: any = { text };

  if (document_id) body.document_id = document_id;
  if (user_id) body.user_id = user_id;

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(body),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

Then you could call it like:

```
await analyzeText(selectedText);
```

or

```
await analyzeText(selectedText, "trustops-handbook-v1", "candidate_1");
```

My recommendation for your interview project

Keep your current version exactly as it is.

It is:

- simple
- deterministic
- easier to demo
- easier to explain

You can say:

> "For the MVP I used static **`document_id`** and **`user_id`** values to simulate a single session context. In a production system these would come from document metadata and authenticated user state."

That's a **very strong product answer**.

One small improvement you *should* make

Add typing to the response so your UI gets autocomplete when you display citations.

Example:

```
interface AnalyzeResponse {
  source_id: string;
  citation_text: string;
  confidence?: number;
  url?: string;
}

export async function analyzeText(text: string): Promise<AnalyzeResponse> {
  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      text,
      document_id: "trustops-handbook-v1",
      user_id: "candidate_1",
    }),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

This will help when you do the **next step**, which is:

displaying `citation_text` in the task pane.

Next milestone (very important)

You now have:

✔️ select text
✔️ click analyze
✔️ POST `/analyze`

The next task should be:

Store the response in React state and render the citation in the UI.

That is the **first visible feature of the real product**.

If you want, I can show you the **best way to implement the citation list UI in ~25 lines**, which will also make the next required feature (jump back to highlighted text) much easier.

> Ok so the front end technically works, however, I tried writing some text and checking what the log was and it has the default source cited. Obviously this makes sense since we put it as a default but how can I remove that and make it apply to my text instead?

What you're seeing is probably **not** that your selected text is being ignored — it's more likely that:

- `document_id` is hardcoded, so the backend is always analyzing against the same source document context, and/or
- the backend is designed to return citations from its fixed indexed document, not from whatever text you typed into Word

So the key distinction is:

- `text` = the passage the user selected in Word
- `document_id` = the source corpus or document the backend searches against

If your backend is built around a fixed source like `"trustops-handbook-v1"`, then it will naturally keep citing from that source, even when the selected text changes.

First: confirm what is actually changing

Before changing anything, log both the selected text and the response:

```
const selectedText = await getSelectedText();
console.log("Selected text being sent:", selectedText);

const result = await analyzeText(selectedText);
console.log("Analyze result:", result);
```

If the selected text changes correctly, then the frontend is doing its job. The repeated source is coming from backend behavior or from the hardcoded `document_id`.

Why this happens

Your current helper sends:

```
body: JSON.stringify({
  text,
  document_id: "trustops-handbook-v1",
  user_id: "candidate_1",
})
```

That means every request says:

> analyze this selected text in the context of **`trustops-handbook-v1`**

So if the backend uses **`document_id`** to decide what source set to search, it will keep returning citations from that document.

What to do instead

Option 1: remove `document_id` and `user_id` entirely

If they are optional, this is the simplest test.

```
export async function analyzeText(text: string) {
  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      text,
    }),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

This lets the backend analyze only based on the text you selected.

Option 2: make them optional parameters

This is cleaner if you may still want them later.

```
interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
) {
  const requestBody: AnalyzeRequest = { text };

  if (document_id) requestBody.document_id = document_id;
  if (user_id) requestBody.user_id = user_id;

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(requestBody),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

Then call it simply as:

```
const result = await analyzeText(selectedText);
```

Important thing to keep in mind

Removing **`document_id`** will only help if the backend actually supports text-only analysis.

If the backend is built like:

- search a known indexed document by **`document_id`**
- return the best citation from that source

then removing it may:

- still return the same source
- fail
- return less useful results

So this is really a backend contract question.

Best way to verify

Try these three tests:

Test A

Send only the selected text:

```
await analyzeText(selectedText);
```

Test B

Send selected text plus current hardcoded **`document_id`**:

```
await analyzeText(selectedText, "trustops-handbook-v1", "candidate_1");
```

Test C

Send very different selected passages and compare the response.

If the citation/source stays nearly identical across all three, then the backend is likely anchored to one indexed source document, which is expected behavior.

Most likely interpretation for your project

Based on the project spec, the add-in is probably supposed to:

- take selected text from Word
- ask the backend to find the best citation **from a predefined source document/system**
- return that citation

So the citation should correspond to the selected text semantically, but the **source repository may still be fixed**.

That means the correct behavior is not “cite my typed text as a new source,” but rather:

> “use my selected text as the query, then find the matching citation from the known source material.”

If that’s the intended design, then keeping a fixed **`document_id`** is actually fine.

What I’d do right now

Change your helper to optional fields so you can test both cases:

```
interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
) {
  const requestBody: AnalyzeRequest = { text };

  if (document_id) requestBody.document_id = document_id;
  if (user_id) requestBody.user_id = user_id;

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(requestBody),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

Then in your button handler:

```
const result = await analyzeText(selectedText);
console.log("Analyze result:", result);
```

If you still get the same source, then the frontend is fine and the backend is behaving according to its own fixed-source logic.

The next thing to inspect would be the actual response payload for 2–3 different text selections to see whether `citation_text` changes even if `source_id` stays the same.

> Ok so I changed them into optional fields. I was given one example of text coming from the default source. When I analyzed on that I got a 95% confidence, but - when I analyze on other random content, I always get the same default values with 82% confidence which is obviously incorrect. Maybe this has to do with the confidence and url being optional? Is there a way to get a sample of the urls or source_ids that are stored in the API? Also, we don't actually have anything implemented to pass in a source_id or User_id

Yeah — this does **not** sound like a frontend bug.

What you're describing strongly suggests the backend is doing one of these:

- searching a **single default source document** and returning its best match every time, or
- returning a **fallback/default citation** when the selected text doesn't match well enough

The spec supports that interpretation. `POST /analyze` takes `text` plus optional `document_id` and `user_id`, and returns a citation object with `source_id`, `citation_text`, optional `confidence`, and optional `url`. Also, `GET /document` is specifically there so you can inspect the backend's source material and validate what

document content is being used for citation analysis.
QSC Take Home Round 2 QSC Take Home Round 2 QSC Take Home Round 2

QSC Take Home Round 2

A key point: **`confidence` and `url` being optional is not the cause of the wrong matching behavior.** Optional just means the backend may omit them from the response. It does **not** change how the backend chooses a source. The matching logic is upstream of that.
QSC Take Home Round 2

Also:

- **`user_id`** is optional and is just an identifier for the current user/test session.
QSC Take Home Round 2
- **`document_id`** is optional but recommended, and identifies the source document being searched against.
QSC Take Home Round 2
- **`source_id`** is **returned by the backend**; it is not something your frontend is supposed to send. The happy path explicitly says the backend returns **`source_id`** and **`citation_text`**.
QSC Take Home Round 2

So your statement is right: you do **not** need to pass in a **`source_id`**. The frontend sends selected text; the backend returns the matched source.
QSC Take Home Round 2

Can you get a sample of stored `urls` or `source_ids`?

From the spec, there is **no endpoint listed for enumerating all source IDs or URLs**. The only documented endpoints are:

- **`GET /health`**
- **`GET /document`**
- **`POST /analyze`**
QSC Take Home Round 2 QSC Take Home Round 2

So:

- you can use **`GET /document`** to inspect the source content the backend is searching against
- you can use repeated **`POST /analyze`** calls to observe which **`source_id`** and **`url`** come back
- but based on the provided instructions, there is **no documented API to list all stored sources**

What I would do next

1. Inspect `GET /document`

This is the most important debugging move right now.

If **`GET /document`** shows a single handbook or a single fixed source corpus, then it makes sense that unrelated text still maps to that same source. The spec literally says **`GET /document`** is for validating what document content is being used for citation analysis.
QSC Take Home Round 2

2. Log the full response for several inputs

Test three categories:

- exact or near-exact text from the example document

- paraphrased text from that same source
- completely unrelated/random text

Then compare:

- `source_id`
- `citation_text`
- `confidence`
- `url`

If unrelated text always returns the same citation at ~0.82 confidence, that looks like a backend fallback/default behavior, not a UI issue.

3. Add frontend handling for weak results

The spec explicitly calls out a failure state for empty/low-confidence responses and suggests showing:

> "Unable to generate citation. Try refining selection."
> QSC Take Home Round 2

So even if the backend returns a citation for garbage text, you can protect the UX by applying a threshold in the frontend. For example:

- if `confidence >= 0.9`, show the citation
- if `confidence < 0.9`, show the "refine selection" message instead

Given your current observations, that would stop the obviously incorrect 0.82 result from being displayed as a valid citation.

Practical recommendation for your project

Keep this mental model:

- `text`: what the user selected in Word
- `document_id`: optional hint about which backend source document to search
- `user_id`: optional session/user tag
- `source_id`: backend output describing what source it matched

So the next best implementation step is:

1. use `GET /document` to understand the backend's actual source material
2. log and compare multiple `/analyze` responses
3. add a low-confidence guard in the UI

A clean rule for the demo would be:

- if there is no result or confidence is below your threshold, show **"Unable to generate citation. Try refining selection."**

- otherwise display the citation

That matches the PRD nicely.

If you want, I'll help you add that confidence-threshold logic directly into your **`AnalyzeText.tsx`** next.

Sources

> sorry I meant to say can we set up something to pass in the document_id and user_id in the request body

Yes — you **can and should** set up your frontend so **`document_id`** and **`user_id`** are passed dynamically rather than hard-coded. This is actually a good architectural improvement and easy to implement.

Right now your helper always sends:

```
{
  text,
  document_id: "trustops-handbook-v1",
  user_id: "candidate_1"
}
```

Instead, make the API helper accept them as parameters and pass them from the UI.

Step 1 — Update your API helper

Modify **`analyzeText()`** to accept the fields.

```
interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
) {
  const requestBody: AnalyzeRequest = { text };

  if (document_id) requestBody.document_id = document_id;
  if (user_id) requestBody.user_id = user_id;

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(requestBody),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

Now the helper is reusable and clean.

Step 2 — Pass the fields from your Analyze component

Inside **`AnalyzeText.tsx`**:

```
const documentId = "trustops-handbook-v1";
const userId = "candidate_1";

const result = await analyzeText(selectedText, documentId, userId);
```

Example inside your handler:

```
const handleAnalyze = async () => {
  try {
    const selectedText = await getSelectedText();

    if (!selectedText) {
      setMessage("Please select text before analyzing.");
      return;
    }

    const documentId = "trustops-handbook-v1";
    const userId = "candidate_1";

    const result = await analyzeText(selectedText, documentId, userId);

    console.log("Analyze result:", result);
  } catch (error) {
    setMessage("Analyze request failed.");
  }
};
```

Step 3 — (Optional but cleaner) store them as constants

At the top of the file:

```
const DOCUMENT_ID = "trustops-handbook-v1";
const USER_ID = "candidate_1";
```

Then call:

```
await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
```

This keeps things tidy.

Why this is better

This lets you easily change the context later:

```
await analyzeText(selectedText, "policy-doc-v2", "candidate_1")
await analyzeText(selectedText, "legal-handbook", "candidate_2")
```

And if the backend later uses these values for indexing or logging, your code is already structured for it.

Important clarification

You **do not need to send `source_id`**.

The workflow is:

```
frontend sends:
  text
  document_id (optional)
  user_id (optional)

backend returns:
  source_id
  citation_text
  confidence
  url
```

So the correct flow is:

```
selected text → analyze → backend chooses source → returns source_id
```

Next feature you should implement

Now that `/analyze` works correctly, the next big step in the project spec is:

display the returned citation in the task pane UI

Specifically:

- store citation results in React state
- render a citation list
- later link it to highlighted text

This is where the add-in actually starts looking like the real product.

If you want, I can also show you the **best way to structure the citation list state now**, because doing it correctly will make the later **"click citation → jump to text"** feature much easier.

okay so a couple things we need to add. First, before we actually go and analyze, I want to add a health check. If the health check passes, we continue as normal, if the health check fails, a specific error message is used and we do not even attempt to analyze

Great idea — adding a **health check gate before analyze** is a very solid engineering decision and aligns with the spec's emphasis on reliability and failure states. It also makes your demo look more production-ready.

The cleanest approach is:

1. Call `checkHealth()`
2. If it fails → show error message and **return early**
3. If it succeeds → continue with `getSelectedText()` and `analyzeText()`

---

1. Ensure you have a health helper in your API file

In `server/api.ts`:

```
export async function checkHealth() {
  const response = await fetch(`${BASE_URL}/health`);

  if (!response.ok) {
    throw new Error("Backend health check failed");
  }

  return response.json();
}
```

2. Import it into `AnalyzeText.tsx`

```
import { analyzeText, checkHealth } from "../server/api";
```

3. Update the Analyze handler

Modify your handler to check backend health **before doing anything else**.

```
const handleAnalyze = async () => {
  try {
    setMessage("");

    // 1️⃣ Check backend health first
    try {
      await checkHealth();
    } catch {
      setMessage("Citation service is currently unavailable. Please try again later.");
      return;
    }

    // 2️⃣ Get selected text
    const selectedText = await getSelectedText();

    if (!selectedText) {
      setMessage("Please select text before analyzing.");
      return;
    }

    // 3️⃣ Call analyze
    const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

    console.log("Analyze result:", result);

  } catch (error) {
    setMessage("Analyze request failed.");
    console.error(error);
  }
};
```

4. Add loading state (recommended)

This prevents repeated clicks and improves UX.

At the top of your component:

```
const [isLoading, setIsLoading] = useState(false);
```

Then update the handler:

```
const handleAnalyze = async () => {
  try {
    setMessage("");
    setIsLoading(true);

    // Health check
    try {
      await checkHealth();
    } catch {
      setMessage("Citation service is currently unavailable.");
      return;
    }

    const selectedText = await getSelectedText();

    if (!selectedText) {
      setMessage("Please select text before analyzing.");
      return;
    }

    const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

    console.log("Analyze result:", result);

  } catch (error) {
    setMessage("Analyze request failed.");
  } finally {
    setIsLoading(false);
  }
};
```

Then update your button:

```
<Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
  {isLoading ? "Analyzing..." : "Analyze"}
</Button>
```

---

5. Resulting user behavior

Case 1 — backend healthy

```
Select text → Analyze → health passes → analyze runs
```

Case 2 — backend down

```
Select text → Analyze → health fails → show error → stop
```

User sees:

```
Citation service is currently unavailable. Please try again later.
```

Case 3 — no text selected

```
Analyze → health passes → selection empty → show message
```

---

6. Why this is a strong implementation

You can explain it like this in the interview:

> "Before sending analysis requests, I implemented a health check against the backend. If the service is unavailable, the UI stops the workflow early and shows a user-friendly message instead of attempting a failing request."

This shows:

- **defensive programming**
- **API reliability awareness**
- **good UX for failure states**

Next feature (very important)

After this step, the next milestone should be:

Render the citation result in the task pane

That means:

- storing `citation_text`, `source_id`, etc. in React state
- displaying them in a list
- preparing to link them back to highlighted text.

If you'd like, I can show you a **simple citation list component that takes ~30 lines and fits perfectly with your current architecture.**

> I just realized getDocument is also never used. Let's implement this in this order: after health check is successful, check if a document_id is being passed to analyzeText. If yes, first check that the document_id being passed to analyzeText is found in the getDocument response. If it is found, proceed with analyzeText. If not, print a specific error message saying there are no records of that document_id. Here is a sample response from getDocument:
>
> "document_id": "trustops-handbook-v1",
> "title": "TrustOps Policy Handbook (Sample)",
> "author": "TrustOps Team",
> "url": "https://example.com/trustops-handbook",
> "date": "2025-05-12",
> "sections": [4 items]

> can you continue

Yes — that’s a good flow, and it gives `getDocument()` a real purpose in the request pipeline.

The sequence becomes:

1. run `checkHealth()`
2. if health fails, stop and show backend-unavailable message
3. if a `document_id` was passed, call `getDocument()`

4. compare the returned `document_id` against the one you plan to send to `analyzeText()`
5. if they match, continue to `analyzeText()`
6. if they do not match, stop and show:
   There are no records of that document_id.
7. if no `document_id` was passed at all, skip the document validation step and continue directly to `analyzeText()`

That is clean, logical, and easy to explain in an interview.

Overall behavior you want

Your Analyze button logic should now behave like this:

```
Analyze clicked
   ↓
health check
   ↓
if unhealthy → stop with service error
   ↓
if document_id exists:
    fetch document metadata
    compare document_id
    if mismatch → stop with document_id error
   ↓
get selected text
   ↓
if no text selected → stop with selection error
   ↓
call analyzeText(...)
   ↓
log/display response
```

That's a strong request pipeline because it validates the environment before making the expensive action.

Important assumption about `getDocument()`

Based on the sample response you shared, `getDocument()` returns a **single document object**, something like:

```
{
  "document_id": "trustops-handbook-v1",
  "title": "TrustOps Policy Handbook (Sample)",
  "author": "TrustOps Team",
  "url": "https://example.com/trustops-handbook",
  "date": "2025-05-12",
  "sections": [...]
}
```

That means you are **not** searching a list of many documents. You are just validating whether the one available backend document matches the `document_id` you intend to use.

So the check is simply:

```
document.document_id === documentId
```

not "look through many records."

Step 1 — make sure `getDocument()` exists in `server/api.ts`

A clean version would be:

```
export interface DocumentResponse {
  document_id: string;
  title: string;
  author: string;
  url: string;
  date: string;
  sections: unknown[];
}

export async function getDocument(): Promise<DocumentResponse> {
  const response = await fetch(`${BASE_URL}/document`);

  if (!response.ok) {
    throw new Error(`Get document request failed with status ${response.status}`);
  }

  return response.json();
}
```

You can loosen **`sections`** later if needed, but **`unknown[]`** is perfectly fine for now since you are not using the actual section data yet.

Step 2 — keep `analyzeText()` parameterized

Your **`analyzeText()`** should stay like this:

```
interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
) {
  const requestBody: AnalyzeRequest = { text };

  if (document_id) requestBody.document_id = document_id;
  if (user_id) requestBody.user_id = user_id;

  const response = await fetch(`${BASE_URL}/analyze`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(requestBody),
  });

  if (!response.ok) {
    throw new Error(`Analyze request failed with status ${response.status}`);
  }

  return response.json();
}
```

This is good because it lets the Analyze component decide whether it wants to pass those values.

Step 3 — add the document validation step in `AnalyzeText.tsx`

Here is the logic you want in the component.

Full example

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";
import { analyzeText, checkHealth, getDocument } from "../server/api";

const DOCUMENT_ID = "trustops-handbook-v1";
const USER_ID = "candidate_1";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
    textAlign: "center",
    maxWidth: "80%",
  },
});

const AnalyzeButton: React.FC = () => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);

  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      setMessage("");
      setIsLoading(true);

      // 1. Health check
      try {
        await checkHealth();
      } catch (error) {
        setMessage("Citation service is currently unavailable. Please try again later.");
        return;
      }

      // 2. If a document_id is being passed, validate it first
      if (DOCUMENT_ID) {
        try {
          const document = await getDocument();

          if (document.document_id !== DOCUMENT_ID) {
            setMessage("There are no records of that document_id.");
            return;
          }
        } catch (error) {
          setMessage("Unable to validate the requested document.");
          return;
        }
      }

      // 3. Get selected text
      const selectedText = await getSelectedText();

      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      // 4. Analyze text
      const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
```

```
      console.log("Analyze result:", result);
    } catch (error) {
      setMessage("Analyze request failed.");
      console.error("Error during analyze flow:", error);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

Why this works well

This implementation separates the validation into distinct stages:

Stage 1: service availability

You don't proceed unless the backend is reachable.

Stage 2: document validity

You only proceed if the document the frontend intends to use is actually recognized by the backend metadata endpoint.

Stage 3: user input validity

You only proceed if the user has selected text.

Stage 4: analysis

Only after all validations pass do you call **`POST /analyze`**.

That is a very good pipeline.

One small design decision: where should `DOCUMENT_ID` and `USER_ID` live?

Right now, if you define them as constants at the top of **`AnalyzeText.tsx`**, that is perfectly fine for your MVP:

```
const DOCUMENT_ID = "trustops-handbook-v1";
const USER_ID = "candidate_1";
```

That keeps the logic simple.

Later, if you want, you can move them into:

- a config file
- component state
- props passed from **`App.tsx`**

But you do not need that yet.

Better version: only validate if document_id is actually present

Since your helper allows **`document_id`** to be optional, you may want to support the case where you intentionally omit it.

For example:

```
const DOCUMENT_ID: string | undefined = "trustops-handbook-v1";
const USER_ID: string | undefined = "candidate_1";
```

Then the validation step becomes meaningful:

```
if (DOCUMENT_ID) {
  const document = await getDocument();

  if (document.document_id !== DOCUMENT_ID) {
    setMessage("There are no records of that document_id.");
    return;
  }
}
```

If **`DOCUMENT_ID`** is **`undefined`**, the app skips that step entirely.

That exactly matches the behavior you described.

Even cleaner: move validation into a helper inside the component

If you want the handler to stay readable, you can isolate the document check:

```
const validateDocumentId = async (documentId?: string): Promise<boolean> => {
  if (!documentId) {
    return true;
  }

  const document = await getDocument();
  return document.document_id === documentId;
};
```

Then use it like:

```
const isValidDocument = await validateDocumentId(DOCUMENT_ID);

if (!isValidDocument) {
  setMessage("There are no records of that document_id.");
  return;
}
```

This makes **`handleAnalyze()`** easier to scan.

Recommended polished version

This is the version I'd actually use, because it reads well and keeps validation logic tidy.

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";
import { analyzeText, checkHealth, getDocument } from "../server/api";

const DOCUMENT_ID: string | undefined = "trustops-handbook-v1";
const USER_ID: string | undefined = "candidate_1";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
    textAlign: "center",
    maxWidth: "80%",
  },
});

const AnalyzeButton: React.FC = () => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);

  const styles = useStyles();

  const validateDocumentId = async (documentId?: string): Promise<boolean> => {
    if (!documentId) {
      return true;
    }

    const document = await getDocument();
    return document.document_id === documentId;
  };

  const handleAnalyze = async () => {
    try {
      setMessage("");
      setIsLoading(true);

      await checkHealth();

      const isValidDocument = await validateDocumentId(DOCUMENT_ID);
      if (!isValidDocument) {
        setMessage("There are no records of that document_id.");
        return;
      }

      const selectedText = await getSelectedText();
      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
      console.log("Analyze result:", result);
    } catch (error) {
      setMessage("Citation service is currently unavailable or the request could not be completed.");
      console.error("Error during analyze flow:", error);
    } finally {
      setIsLoading(false);
    }
  };
```

```
  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

Optional refinement: distinguish document validation errors from service errors

Right now, the catch block can make all failures look similar. If you want clearer UX, you can split them more explicitly.

For example:

```
const handleAnalyze = async () => {
  setMessage("");
  setIsLoading(true);

  try {
    await checkHealth();
  } catch (error) {
    setMessage("Citation service is currently unavailable. Please try again later.");
    setIsLoading(false);
    return;
  }

  try {
    const isValidDocument = await validateDocumentId(DOCUMENT_ID);
    if (!isValidDocument) {
      setMessage("There are no records of that document_id.");
      setIsLoading(false);
      return;
    }
  } catch (error) {
    setMessage("Unable to validate the requested document.");
    setIsLoading(false);
    return;
  }

  try {
    const selectedText = await getSelectedText();
    if (!selectedText) {
      setMessage("Please select text before analyzing.");
      setIsLoading(false);
      return;
    }

    const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
    console.log("Analyze result:", result);
  } catch (error) {
    setMessage("Analyze request failed.");
    console.error(error);
  } finally {
    setIsLoading(false);
  }
};
```

This version is a bit longer, but it gives more specific messages at each stage.

Practical note about `getDocument()`

Since **`getDocument()`** appears to return only one document, the validation is basically checking whether the frontend's configured **`DOCUMENT_ID`** matches the backend's currently available document.

So if you set:

```
const DOCUMENT_ID = "wrong-id";
```

you should expect:

There are no records of that document_id.

If you set:

```
const DOCUMENT_ID = "trustops-handbook-v1";
```

the validation should pass.

That's a good quick test to run manually.

What to test after implementing this

Run these four cases:

Case 1 — health fails

Expected:

- message says citation service unavailable
- no attempt to analyze

Case 2 — health passes, bad document id

Set:

```
const DOCUMENT_ID = "fake-doc-id";
```

Expected:

- message says there are no records of that document_id
- no attempt to analyze

Case 3 — health passes, valid document id, no selection

Expected:

- message says please select text before analyzing

Case 4 — all validations pass

Expected:

- analyze request runs

- result logs successfully

Best next step after this

Once this validation pipeline works, the next thing to add should be:

- store the analyze result in state
- render the returned `citation_text`, `source_id`, maybe confidence
- stop relying only on console logs

That's the point where the add-in starts looking like the actual required MVP rather than just a request validator.

Send me your current `AnalyzeText.tsx` and I can plug this flow directly into your exact file structure.

> ok another error I want to add for analyzeText - if it doesn't complete within 5 seconds, it should return a timeout error and print a specific error message

To enforce a **5-second timeout** for `analyzeText`, the best approach is to use `AbortController` with `fetch`. This allows you to cancel the request if it takes too long and return a specific timeout error.

This should be implemented **inside your API helper**, not in the UI component, because timeout logic belongs to the network layer.

---

1. Update `analyzeText()` to support a timeout

Modify your helper in `server/api.ts`.

```
interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 5000);

  const requestBody: AnalyzeRequest = { text };

  if (document_id) requestBody.document_id = document_id;
  if (user_id) requestBody.user_id = user_id;

  try {
    const response = await fetch(`${BASE_URL}/analyze`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(requestBody),
      signal: controller.signal,
    });

    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`Analyze request failed with status ${response.status}`);
    }

    return response.json();
  } catch (error: any) {
    if (error.name === "AbortError") {
      throw new Error("Analyze request timed out after 5 seconds.");
    }

    throw error;
  }
}
```

---

2. What this does

The flow becomes:

```
send POST /analyze
      ↓
start 5 second timer
      ↓
if response returns before timeout → clear timer → continue
      ↓
if 5 seconds pass → AbortController cancels request
      ↓
throw timeout error
```

---

3. Handle the timeout in `AnalyzeText.tsx`

Now update the handler so it shows a **specific message** when a timeout occurs.

Example:

```
try {
  const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
  console.log("Analyze result:", result);
} catch (error: any) {

  if (error.message.includes("timed out")) {
    setMessage("Citation analysis timed out. Please try again.");
  } else {
    setMessage("Analyze request failed.");
  }

  console.error(error);
}
```

---

4. Resulting behavior

Normal request (<5s)

```
Analyze clicked
↓
POST /analyze
↓
response received
↓
display citation
```

Slow backend (>5s)

```
Analyze clicked
↓
POST /analyze
↓
5 seconds pass
↓
request aborted
↓
timeout message shown
```

User sees:

```
Citation analysis timed out. Please try again.
```

---

5. Why this is the correct place to implement timeout

Putting the timeout inside the **API helper**:

- keeps UI components simple
- centralizes network behavior
- allows reuse if you call `/analyze` elsewhere

This is exactly how production frontends handle slow APIs.

---

6. Optional improvement (recommended)

You can make the timeout **configurable**.

```
const ANALYZE_TIMEOUT = 5000;
```

Then:

```
const timeoutId = setTimeout(() => controller.abort(), ANALYZE_TIMEOUT);
```

This avoids hardcoding values in multiple places.

---

7. Your analyze pipeline now looks like this

Your request logic is now very robust:

```
Analyze button clicked
      ↓
checkHealth()
      ↓
validate document_id via getDocument()
      ↓
getSelectedText()
      ↓
POST /analyze (5 second timeout)
      ↓
display citation OR error message
```

This is **exactly the type of defensive workflow interviewers like to see**.

Next important feature (coming soon)

After this timeout logic, the next real product milestone should be:

Displaying the citation results in the task pane UI instead of only logging them.

That requires:

- React state for citation results
- a citation list component
- storing the mapping between citation and selected text

This step unlocks the later **"click citation → jump back to highlighted text"** feature.

If you'd like, I can also show you the **cleanest citation list implementation (about 35 lines)** that fits perfectly with your current code.

> now the next error i want to handle with analyzeText is if there is a low confidence score returned from the analyze endpoint. I'm setting the threshold as 0.6 - anything under this threshold should get an error message printed saying there was no matching source

The best place to enforce a **low-confidence threshold** is inside the **`analyzeText()`** helper itself, just like you did with the timeout. That keeps the API layer responsible for validating the response before the UI ever receives it.

The idea is:

1. Send the **`/analyze`** request

2. Parse the response
3. Check `confidence`
4. If it exists and is **below 0.6**, throw an error
5. Otherwise return the result normally

This keeps the component logic simple and ensures every caller gets validated data.

---

1. Update `analyzeText()` to enforce a confidence threshold

In `server/api.ts`:

```
const ANALYZE_TIMEOUT = 5000;
const CONFIDENCE_THRESHOLD = 0.6;

interface AnalyzeRequest {
  text: string;
  document_id?: string;
  user_id?: string;
}

interface AnalyzeResponse {
  source_id: string;
  citation_text: string;
  confidence?: number;
  url?: string;
}

export async function analyzeText(
  text: string,
  document_id?: string,
  user_id?: string
): Promise<AnalyzeResponse> {

  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), ANALYZE_TIMEOUT);

  const requestBody: AnalyzeRequest = { text };

  if (document_id) requestBody.document_id = document_id;
  if (user_id) requestBody.user_id = user_id;

  try {
    const response = await fetch(`${BASE_URL}/analyze`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(requestBody),
      signal: controller.signal,
    });

    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`Analyze request failed with status ${response.status}`);
    }

    const data: AnalyzeResponse = await response.json();

    // Low confidence check
    if (data.confidence !== undefined && data.confidence < CONFIDENCE_THRESHOLD) {
      throw new Error("NO_MATCHING_SOURCE");
    }

    return data;

  } catch (error: any) {

    if (error.name === "AbortError") {
      throw new Error("ANALYZE_TIMEOUT");
    }

    throw error;
  }
}
```

---

2. Handle the new error in `AnalyzeText.tsx`

Your UI should interpret the error and show the correct message.

Example:

```
try {
  const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
  console.log("Analyze result:", result);

} catch (error: any) {

  if (error.message === "ANALYZE_TIMEOUT") {
    setMessage("Citation analysis timed out. Please try again.");

  } else if (error.message === "NO_MATCHING_SOURCE") {
    setMessage("No matching source was found for the selected text.");

  } else {
    setMessage("Analyze request failed.");
  }

  console.error(error);
}
```

---

3. Resulting behavior

Valid citation

```
confidence = 0.92
```

Result:

```
citation displayed normally
```

Weak match

```
confidence = 0.41
```

Result:

```
No matching source was found for the selected text.
```

Timeout

```
request > 5 seconds
```

Result:

```
Citation analysis timed out. Please try again.
```

Backend failure

```
HTTP 500
```

Result:

```
Analyze request failed.
```

---

4. Why this design is good

Putting the confidence logic inside the API helper:

- keeps UI code simpler

- ensures **all callers enforce the threshold**
- avoids accidentally displaying weak matches

Your network layer is now responsible for validating:

```
timeout
HTTP failure
low confidence
```

while the UI simply reacts to the error state.

---

5. Your full analyze pipeline now looks like this

```
Analyze button clicked
        ↓
checkHealth()
        ↓
validate document_id with getDocument()
        ↓
getSelectedText()
        ↓
POST /analyze
        ↓
timeout check (5s)
        ↓
confidence threshold check (<0.6)
        ↓
return valid citation
```

This is a **very robust request flow** and exactly the type of defensive architecture interviewers like to see.

Next milestone (very important)

At the moment your results are still being **logged to the console**.

The next step that will make the add-in feel like a real product is:

- storing the `AnalyzeResponse` in React state
- displaying `citation_text`, `source_id`, and optionally `url`
- rendering a **citation list component**

That also sets up the later requirement of:

```
click citation → jump to highlighted text
```

which depends on storing citation metadata.

> okay I want to change this so that it throws an error if the confidence is less than 0.6 OR if the confidence is equal to 0.82 exactly
>
> if (data.confidence !== undefined && data.confidence < CONFIDENCE_THRESHOLD) {

You just need to extend the condition so it checks **both cases**:

- confidence **< 0.6**
- confidence **=== 0.82**

You can combine them with a logical OR.

Updated condition

```
if (
  data.confidence !== undefined &&
  (data.confidence < CONFIDENCE_THRESHOLD || data.confidence === 0.82)
) {
  throw new Error("NO_MATCHING_SOURCE");
}
```

Explanation

The logic now reads as:

```
If confidence exists AND
   (confidence < 0.6 OR confidence == 0.82)
then throw an error
```

So the following values will trigger the error:

Confidence	Result
0.95	valid
0.61	valid
0.59	error
0.41	error
0.82	error

Important note about floating point values

Sometimes APIs return floating values like:

```
0.8200000001
```

or

```
0.8199999999
```

If you want to be **extra safe**, use a tolerance check:

```
const EPSILON = 0.0001;

if (
  data.confidence !== undefined &&
  (
    data.confidence < CONFIDENCE_THRESHOLD ||
    Math.abs(data.confidence - 0.82) < EPSILON
  )
) {
  throw new Error("NO_MATCHING_SOURCE");
}
```

This prevents tiny floating point differences from bypassing the check.

Your final validation block might look like

```
const data: AnalyzeResponse = await response.json();

const EPSILON = 0.0001;

if (
  data.confidence !== undefined &&
  (
    data.confidence < CONFIDENCE_THRESHOLD ||
    Math.abs(data.confidence - 0.82) < EPSILON
  )
) {
  throw new Error("NO_MATCHING_SOURCE");
}

return data;
```

Resulting behavior

Your analyze request will now fail if:

- the backend returns a weak match (`confidence < 0.6`)
- the backend returns the **default fallback result** (`confidence = 0.82`)

Which matches the behavior you observed from the API.

If you'd like, the **next thing I strongly recommend implementing** is a **single centralized error enum for your API layer** (`TIMEOUT`, `NO_MATCH`, `HTTP_ERROR`, etc.). It will make your error handling in `AnalyzeText.tsx` much cleaner and easier to explain in your interview.

> Awesome! We've officially handled all the edge cases. Next, we want to render the citations to the user. The first step to that is highlighting the text the user selected that was sent to the backend to generate the citation

Perfect — this is the right next step.

Now that your analyze flow is validated, the next milestone is to **visually mark the selected text in the Word document** after a successful analyze call. That gives the user immediate feedback that:

- their selection was captured
- that exact passage is now linked to a citation
- the add-in is starting to feel like the real MVP

The best way to do this is to add a new Office.js helper in `taskpane.ts`, similar to `getSelectedText()`.

What to implement

After `analyzeText()` succeeds:

1. get the current Word selection
2. apply a highlight color to that selected range
3. sync the document

This should happen **only after** the analyze request succeeds, not before. That way you don't highlight text if the request fails.

Step 1 — add a highlight helper in `taskpane.ts`

Since you're keeping your Word helpers in **`taskpane.ts`**, add a function like this:

```
export async function highlightSelectedText(): Promise<void> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();
    selection.font.highlightColor = "#FFF59D";

    await context.sync();
  });
}
```

Why this works

- **`context.document.getSelection()`** gets the current selected range in Word
- **`selection.font.highlightColor`** applies a highlight to that range
- **`await context.sync()`** commits the change to the document

The color **`#FFF59D`** is a light yellow that usually works well as a citation highlight. You can change it later.

Step 2 — import it into `AnalyzeText.tsx`

At the top of your file, where you already import **`getSelectedText`**, add **`highlightSelectedText`** too.

For example:

```
import { getSelectedText, highlightSelectedText } from "../taskpane";
```

Adjust the path only if needed, but based on your current setup this should likely be right.

Step 3 — call it after analyze succeeds

Inside your analyze handler, after this line:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
```

add:

```
await highlightSelectedText();
```

So the main success flow becomes:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
await highlightSelectedText();
console.log("Analyze result:", result);
```

Example success section in `AnalyzeText.tsx`

Here's the exact part that changes:

```
try {
  const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
  await highlightSelectedText();

  console.log("Analyze result:", result);
} catch (error: any) {
  if (error.message === "ANALYZE_TIMEOUT") {
    setMessage("Citation analysis timed out. Please try again.");
  } else if (error.message === "NO_MATCHING_SOURCE") {
    setMessage("No matching source was found for the selected text.");
  } else {
    setMessage("Analyze request failed.");
  }

  console.error(error);
}
```

Important note about selection timing

This works best if the current Word selection is still active when you call **`highlightSelectedText()`**.

In your current flow, that should be fine because:

- user selects text
- clicks Analyze
- backend runs
- after success, you highlight the selection

But there is one subtle risk:
if the user changes the selection while the analyze request is in flight, **`getSelection()`** may no longer point to the original text.

For your MVP, this is usually acceptable. But the safer version is to:

- capture the range at the moment Analyze is clicked
- store or re-use that same range later

That is a more advanced step. For now, your current approach is good enough and fits the MVP stage.

Recommended helper name

Use:

```
highlightSelectedText()
```

That name is clear and future-proof.

Later, if you want to support re-highlighting arbitrary ranges or removing highlight, you can add:

- `clearSelectedTextHighlight()`
- `highlightRange()`

But not needed yet.

Suggested updated `taskpane.ts`

If you want it to stay organized, your `taskpane.ts` can now have three exported helpers:

```
export async function insertText(text: string) {
  return Word.run(async (context) => {
    const range = context.document.getSelection();
    range.insertText(text, Word.InsertLocation.replace);
    await context.sync();
  });
}

export async function getSelectedText(): Promise<string> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();
    selection.load("text");

    await context.sync();

    return selection.text.trim();
  });
}

export async function highlightSelectedText(): Promise<void> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();
    selection.font.highlightColor = "#FFF59D";

    await context.sync();
  });
}
```

That keeps all your Word document operations together.

Optional improvement: use a constant for the highlight color

If you want this a bit cleaner:

```
const CITATION_HIGHLIGHT_COLOR = "#FFF59D";

export async function highlightSelectedText(): Promise<void> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();
    selection.font.highlightColor = CITATION_HIGHLIGHT_COLOR;

    await context.sync();
  });
}
```

This is nice if you later decide to change the style globally.

What the flow becomes now

Your analyze pipeline is now:

```
click Analyze
→ health check
→ validate document_id
→ get selected text
→ call /analyze
→ if success, highlight selected text
→ log returned citation
```

That's a strong next step because now the user sees a visible document-side effect.

What to test right after adding this

Test these cases:

Case 1 — successful analyze

Expected:

- no error message
- selected text becomes highlighted
- analyze result logs successfully

Case 2 — no selection

Expected:

- no highlight applied
- "Please select text before analyzing."

Case 3 — backend failure / timeout / low confidence

Expected:

- no highlight applied
- correct error message shown

This is important because the highlight should only happen on a successful citation generation.

What comes immediately after highlighting

Once highlighting works, the next step is:

store the returned citation in React state and render it in the task pane

That should include at least:

- **`citation_text`**
- **`source_id`**
- maybe **`confidence`**

- maybe `url`

And ideally, when you store that citation, you also store some link to the highlighted text for the later "click citation → jump back to text" feature.

So the natural next sequence is:

1. highlight selected text
2. save the citation result in state
3. render citation cards/list items in the UI
4. later add click-to-jump behavior

If you want, I can help you build the next part cleanly: a **citation state shape plus a simple citation list UI** that fits into your current `App.tsx` structure.

> Okay now the next thing is we need to make a scrolling pane that all the references go into. Additionally, we want to keep the link between each reference and the text that was highlighted to select it

Yes — this is the point where you should move from a "button action" to actual **citation state** in the UI.

You now need two things working together:

1. a **scrollable citations pane**
2. a **mapping between each citation and the selected text/range it came from**

The cleanest MVP approach is:

- keep the **citation list state in `App.tsx`**
- let `AnalyzeText.tsx` create a new citation entry after successful analyze
- render the entries in a separate scrollable component like `CitationList.tsx`
- store some identifier or selection metadata with each citation so later you can click the citation and re-select the text

Because Word range persistence is tricky, the safest MVP is to store:

- the returned citation data
- the selected text string
- a generated local id
- optionally a timestamp

That gives you a good session-only mapping even before you implement jump-back behavior.

Recommended component structure

Your files can look like this:

```
App.tsx
AnalyzeText.tsx
CitationList.tsx
TextInsertion.tsx
```

And the flow becomes:

```
App.tsx
  ├── AnalyzeText.tsx
  └── CitationList.tsx
```

`App.tsx` owns the citations state and passes:

- an **`onCitationCreated`** callback into **`AnalyzeText`**
- the **`citations`** array into **`CitationList`**

Step 1 — define a citation type

Put this in **`App.tsx`** for now, or in a shared types file later.

```
export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}
```

This is the minimum useful structure.

Why this works

- **`id`** gives each citation a stable local key
- **`selectedText`** links the citation to the text that was highlighted
- **`sourceId`**, **`citationText`**, **`confidence`**, **`url`** come from the backend

Step 2 — move citation state into `App.tsx`

Update **`App.tsx`** so it stores all citations.

Example `App.tsx`

```
import * as React from "react";
import { useState } from "react";
import Header from "./Header";
import HeroList, { HeroListItem } from "./HeroList";
import TextInsertion from "./TextInsertion";
import AnalyzeButton from "./AnalyzeText";
import CitationList from "./CitationList";
import { makeStyles } from "@fluentui/react-components";
import { Ribbon24Regular, LockOpen24Regular, DesignIdeas24Regular } from "@fluentui/react-icons";
import { insertText } from "../taskpane";

export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}

interface AppProps {
  title: string;
}

const useStyles = makeStyles({
  root: {
    minHeight: "100vh",
    paddingBottom: "24px",
  },
});

const App: React.FC<AppProps> = (props: AppProps) => {
  const styles = useStyles();
  const [citations, setCitations] = useState<CitationItem[]>([]);

  const listItems: HeroListItem[] = [
    {
      icon: <Ribbon24Regular />,
      primaryText: "Achieve more with Office integration",
    },
    {
      icon: <LockOpen24Regular />,
      primaryText: "Unlock features and functionality",
    },
    {
      icon: <DesignIdeas24Regular />,
      primaryText: "Create and visualize like a pro",
    },
  ];

  const handleCitationCreated = (newCitation: CitationItem) => {
    setCitations((prevCitations) => [newCitation, ...prevCitations]);
  };

  return (
    <div className={styles.root}>
      <Header logo="assets/logo-filled.png" title={props.title} message="Welcome" />
      <HeroList message="Discover what this add-in can do for you today!" items={listItems} />
      <TextInsertion insertText={insertText} />
      <AnalyzeButton onCitationCreated={handleCitationCreated} />
      <CitationList citations={citations} />
    </div>
  );
};

export default App;
```

Step 3 — update `AnalyzeText.tsx` to send new citations upward

Your Analyze component should no longer just log results.
It should:

- get selected text
- analyze it
- highlight it
- create a citation object
- call **`onCitationCreated(...)`**

Update props

```
interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}

interface AnalyzeButtonProps {
  onCitationCreated: (citation: CitationItem) => void;
}
```

Example `AnalyzeText.tsx`

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText, highlightSelectedText } from "../taskpane";
import { analyzeText, checkHealth, getDocument } from "../server/api";

interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}

interface AnalyzeButtonProps {
  onCitationCreated: (citation: CitationItem) => void;
}

const DOCUMENT_ID: string | undefined = "trustops-handbook-v1";
const USER_ID: string | undefined = "candidate_1";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
    textAlign: "center",
    maxWidth: "80%",
  },
});

const AnalyzeButton: React.FC<AnalyzeButtonProps> = ({ onCitationCreated }) => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);

  const styles = useStyles();

  const validateDocumentId = async (documentId?: string): Promise<boolean> => {
    if (!documentId) {
      return true;
    }

    const document = await getDocument();
    return document.document_id === documentId;
  };

  const handleAnalyze = async () => {
    try {
      setMessage("");
      setIsLoading(true);

      await checkHealth();

      const isValidDocument = await validateDocumentId(DOCUMENT_ID);
      if (!isValidDocument) {
        setMessage("There are no records of that document_id.");
        return;
      }

      const selectedText = await getSelectedText();
      if (!selectedText) {
        setMessage("Please select text before analyzing.");
```

```
        return;
      }

      const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

      await highlightSelectedText();

      const newCitation: CitationItem = {
        id: crypto.randomUUID(),
        selectedText,
        sourceId: result.source_id,
        citationText: result.citation_text,
        confidence: result.confidence,
        url: result.url,
      };

      onCitationCreated(newCitation);
    } catch (error: any) {
      if (error.message === "ANALYZE_TIMEOUT") {
        setMessage("Citation analysis timed out. Please try again.");
      } else if (error.message === "NO_MATCHING_SOURCE") {
        setMessage("No matching source was found for the selected text.");
      } else {
        setMessage("Analyze request failed.");
      }

      console.error("Error during analyze flow:", error);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

Step 4 — create the scrollable `CitationList.tsx`

This is the pane where all references appear.

Example `CitationList.tsx`

```
import * as React from "react";
import { makeStyles, tokens } from "@fluentui/react-components";

interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}

interface CitationListProps {
  citations: CitationItem[];
}

const useStyles = makeStyles({
  container: {
    marginTop: "30px",
    marginLeft: "20px",
    marginRight: "20px",
  },
  title: {
    fontWeight: tokens.fontWeightSemibold,
    marginBottom: "12px",
  },
  scrollPane: {
    maxHeight: "300px",
    overflowY: "auto",
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
    backgroundColor: tokens.colorNeutralBackground1,
  },
  emptyState: {
    color: tokens.colorNeutralForeground3,
  },
  citationCard: {
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
    marginBottom: "12px",
    backgroundColor: tokens.colorNeutralBackground2,
  },
  label: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "6px",
    marginBottom: "4px",
  },
  text: {
    wordBreak: "break-word",
  },
  link: {
    color: tokens.colorBrandForegroundLink,
    textDecoration: "none",
  },
});

const CitationList: React.FC<CitationListProps> = ({ citations }) => {
  const styles = useStyles();

  return (
    <div className={styles.container}>
      <div className={styles.title}>References</div>

      <div className={styles.scrollPane}>
        {citations.length === 0 ? (
          <div className={styles.emptyState}>No references yet.</div>
        ) : (
          citations.map((citation) => (
            <div key={citation.id} className={styles.citationCard}>
              <div className={styles.label}>Citation</div>
              <div className={styles.text}>{citation.citationText}</div>

              <div className={styles.label}>Source ID</div>
```

```
                <div className={styles.text}>{citation.sourceId}</div>

                <div className={styles.label}>Selected Text</div>
                <div className={styles.text}>{citation.selectedText}</div>

                {citation.confidence !== undefined && (
                  <>
                    <div className={styles.label}>Confidence</div>
                    <div className={styles.text}>{citation.confidence}</div>
                  </>
                )}

                {citation.url && (
                  <>
                    <div className={styles.label}>URL</div>
                    <a
                      href={citation.url}
                      target="_blank"
                      rel="noreferrer"
                      className={styles.link}
                    >
                      {citation.url}
                    </a>
                  </>
                )}
              </div>
            ))
          )}
        </div>
      </div>
    );
};

export default CitationList;
```

What this gives you right away

Once you do this, every successful Analyze action will:

- highlight the selected text in Word
- create a citation object
- insert it at the top of the list
- show all previous references in a scrollable pane

That satisfies the “references go into a pane” part very well.

Keeping the link between citation and highlighted text

For the MVP, the link is the **`selectedText`** stored inside each citation item:

```
selectedText: string;
```

That means each reference already knows:

- which text was selected
- which citation was generated from it

This is the **simplest session-only mapping**.

Why this is acceptable for now

Word range objects are not ideal to keep directly in React state. They're tied to Office.js request contexts and can become unreliable across async calls.

So for now, storing:

- **`id`**
- **`selectedText`**
- citation metadata

is the safest design.

Better mapping for the next step

When you implement **click citation → re-select text**, you'll likely want to enhance each citation item with something like:

```
rangeText: string;
```

which is basically the same as **`selectedText`**, or maybe:

```
selectedTextPreview: string;
```

Then when the user clicks the reference, you can search the document for that text and select the first match.

That means your current structure already sets you up for the next feature.

Optional improvement: add a timestamp

You may want to know when each reference was created.

```
createdAt: string;
```

Example:

```
createdAt: new Date().toISOString(),
```

This can help with ordering or display later, but it's optional for now.

Even better type sharing

Right now I repeated **`CitationItem`** in multiple files for clarity.

A cleaner version is to create a shared file:

```
types.ts
```

with:

```
export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}
```

Then import it into both `App.tsx` and `CitationList.tsx` and `AnalyzeText.tsx`.

That reduces duplication.

Recommended order to implement

Do it in this order:

1. create `CitationList.tsx`
2. add `citations` state to `App.tsx`
3. pass `onCitationCreated` into `AnalyzeText`
4. create a new citation object after successful analyze
5. render it in the scrolling pane
6. verify multiple citations appear and the pane scrolls

What to test

Test 1 — first successful analyze

Expected:

- selected text highlighted
- one citation appears in the References pane

Test 2 — second successful analyze

Expected:

- second citation appears
- first one remains
- newest citation appears at top

Test 3 — failed analyze

Expected:

- no new citation added
- error message shown
- pane remains unchanged

Test 4 — long text

Expected:

- selected text and citation wrap properly
- pane scrolls instead of expanding forever

Best next step after this

Once this is in place, the next feature is:

make each reference clickable and re-select the linked text in the document

Since you're already storing `selectedText`, the easiest implementation will be:

- click citation
- search document for `selectedText`
- select the first match

That's the natural next move after the scrollable pane is done.

Paste your current `AnalyzeText.tsx` and `App.tsx` once you wire the state in, and I can help you connect the props and types exactly.

> Can you keep AnalyzeText.tsx as close as possible to my initial format

Yes — keep `AnalyzeText.tsx` simple and close to your `TextInsertion.tsx` pattern.

That means:

- keep styles inside the file
- keep `useState` inside the file
- keep one main handler like `handleAnalyze`
- receive a prop callback from `App.tsx`
- avoid over-abstracting

The main change is just that instead of calling `props.insertText(text)`, this component will:

- validate
- call analyze
- highlight selected text
- pass a citation object up through props

`AnalyzeText.tsx`

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, tokens, makeStyles } from "@fluentui/react-components";
import { getSelectedText, highlightSelectedText } from "../taskpane";
import { analyzeText, checkHealth, getDocument } from "../server/api";

/* global crypto */

export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}

interface AnalyzeTextProps {
  onCitationCreated: (citation: CitationItem) => void;
}

const DOCUMENT_ID: string | undefined = "trustops-handbook-v1";
const USER_ID: string | undefined = "candidate_1";

const useStyles = makeStyles({
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
    textAlign: "center",
    maxWidth: "80%",
  },
});

const AnalyzeText: React.FC<AnalyzeTextProps> = (props: AnalyzeTextProps) => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);

  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      setMessage("");
      setIsLoading(true);

      await checkHealth();

      if (DOCUMENT_ID) {
        const document = await getDocument();

        if (document.document_id !== DOCUMENT_ID) {
          setMessage("There are no records of that document_id.");
          return;
        }
      }

      const selectedText = await getSelectedText();

      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
```

```
      await highlightSelectedText();

      const newCitation: CitationItem = {
        id: crypto.randomUUID(),
        selectedText,
        sourceId: result.source_id,
        citationText: result.citation_text,
        confidence: result.confidence,
        url: result.url,
      };

      props.onCitationCreated(newCitation);
    } catch (error: any) {
      if (error.message === "ANALYZE_TIMEOUT") {
        setMessage("Citation analysis timed out. Please try again.");
      } else if (error.message === "NO_MATCHING_SOURCE") {
        setMessage("There was no matching source.");
      } else {
        setMessage("Analyze request failed.");
      }

      console.error("Error during analyze flow:", error);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" disabled={isLoading} size="large" onClick={handleAnalyze}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeText;
```

CitationList.tsx

This gives you the scrollable references pane while staying straightforward.

```
import * as React from "react";
import { makeStyles, tokens } from "@fluentui/react-components";
import type { CitationItem } from "./AnalyzeText";

interface CitationListProps {
  citations: CitationItem[];
}

const useStyles = makeStyles({
  container: {
    marginTop: "30px",
    marginLeft: "20px",
    marginRight: "20px",
  },
  title: {
    fontWeight: tokens.fontWeightSemibold,
    marginBottom: "10px",
  },
  scrollPane: {
    maxHeight: "300px",
    overflowY: "auto",
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
  },
  emptyState: {
    color: tokens.colorNeutralForeground3,
  },
  citationCard: {
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
    marginBottom: "12px",
    backgroundColor: tokens.colorNeutralBackground2,
  },
  label: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "6px",
    marginBottom: "4px",
  },
  text: {
    wordBreak: "break-word",
  },
});

const CitationList: React.FC<CitationListProps> = (props: CitationListProps) => {
  const styles = useStyles();

  return (
    <div className={styles.container}>
      <div className={styles.title}>References</div>

      <div className={styles.scrollPane}>
        {props.citations.length === 0 ? (
          <div className={styles.emptyState}>No references yet.</div>
        ) : (
          props.citations.map((citation) => (
            <div key={citation.id} className={styles.citationCard}>
              <div className={styles.label}>Citation</div>
              <div className={styles.text}>{citation.citationText}</div>

              <div className={styles.label}>Source ID</div>
              <div className={styles.text}>{citation.sourceId}</div>

              <div className={styles.label}>Selected Text</div>
              <div className={styles.text}>{citation.selectedText}</div>
            </div>
          ))
        )}
      </div>
    </div>
  );
};

export default CitationList;
```

App.tsx

This is the minimum change needed to support the references pane.

```
import * as React from "react";
import { useState } from "react";
import Header from "./Header";
import HeroList, { HeroListItem } from "./HeroList";
import TextInsertion from "./TextInsertion";
import AnalyzeText, { CitationItem } from "./AnalyzeText";
import CitationList from "./CitationList";
import { makeStyles } from "@fluentui/react-components";
import { Ribbon24Regular, LockOpen24Regular, DesignIdeas24Regular } from "@fluentui/react-icons";
import { insertText } from "../taskpane";

interface AppProps {
  title: string;
}

const useStyles = makeStyles({
  root: {
    minHeight: "100vh",
  },
});

const App: React.FC<AppProps> = (props: AppProps) => {
  const styles = useStyles();
  const [citations, setCitations] = useState<CitationItem[]>([]);

  const listItems: HeroListItem[] = [
    {
      icon: <Ribbon24Regular />,
      primaryText: "Achieve more with Office integration",
    },
    {
      icon: <LockOpen24Regular />,
      primaryText: "Unlock features and functionality",
    },
    {
      icon: <DesignIdeas24Regular />,
      primaryText: "Create and visualize like a pro",
    },
  ];

  const handleCitationCreated = (citation: CitationItem) => {
    setCitations((prevCitations) => [citation, ...prevCitations]);
  };

  return (
    <div className={styles.root}>
      <Header logo="assets/logo-filled.png" title={props.title} message="Welcome" />
      <HeroList message="Discover what this add-in can do for you today!" items={listItems} />
      <TextInsertion insertText={insertText} />
      <AnalyzeText onCitationCreated={handleCitationCreated} />
      <CitationList citations={citations} />
    </div>
  );
};

export default App;
```

Why this stays close to your original format

`AnalyzeText.tsx` now matches your **`TextInsertion.tsx`** style pretty closely:

- local **`useState`**

- local `useStyles`
- one prop interface
- one main handler
- simple JSX with `Field` + `Button`

So it won't feel like a big architectural jump.

The next step after this is making each citation clickable so it re-selects the highlighted text.

No - for AnalyzeText.tsx this is my current code, follow the same general order of my code in the new script that has the citations implemented:

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";
import { analyzeText, checkHealth, getDocument, highlightSelectedText } from "../../services/api";

const DOCUMENT_ID = "trustops-handbook-v1";
const USER_ID = "candidate_1";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
  },
});

const AnalyzeButton: React.FC = () => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      setMessage("");
```

```
    setIsLoading(true);

    // Check API connection first using health endpoint
    try {
      await checkHealth();
    } catch {
      setMessage("Failed to connect to citation service. Please try again later.");
      return;
    }

    // Check document_id exists if a document_id is being passed to analyzeText
    if(DOCUMENT_ID) {
      try {
        const document = await getDocument();

        if (document.document_id !== DOCUMENT_ID) {
          setMessage("No information found on requested document_id.");
          return;
        }
      } catch (error) {
        setMessage("Unable to validate the requested document. Please try again.");
        return;
      }
    }

    // Get selected text
    const selectedText = await getSelectedText();
    // Handle no text selected
    if (!selectedText) {
      setMessage("Please select text before analyzing.");
      return;
    }

    // Analyze selected text
    const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
    await highlightSelectedText();
    console.log("Analyze result:", result);  // take this out later
  } catch (error: any) {
    if (error.message === "ANALYZE_TIMEOUT") {
      setMessage("Citation analysis timed out. Please try again.");
    } else if (error.message === "NO_MATCHING_SOURCE") {
      setMessage("Unable to generate citation. Try refining selection.");
    } else {
      setMessage("Analyze request failed.");
    }
    console.error("Error analyzing text:", error);
  } finally {
    setIsLoading(false);
  }
};
```

```
  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

Yes — here's a version that keeps the **same overall structure, order, and style** as your current `AnalyzeText.tsx`, while adding the citation creation logic.

The only real additions are:

- a `CitationItem` interface
- a props interface with `onCitationCreated`
- building a citation object after a successful analyze
- passing that citation up to `App.tsx`

Updated `AnalyzeText.tsx`

```
import * as React from "react";
import { useState } from "react";
import { Button, Field, makeStyles, tokens } from "@fluentui/react-components";
import { getSelectedText } from "../taskpane";
import { analyzeText, checkHealth, getDocument, highlightSelectedText } from "../../services/api";

/* global crypto */

export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
}

interface AnalyzeButtonProps {
  onCitationCreated: (citation: CitationItem) => void;
}

const DOCUMENT_ID = "trustops-handbook-v1";
const USER_ID = "candidate_1";

const useStyles = makeStyles({
  analyzeContainer: {
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    marginTop: "30px",
  },
  instructions: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "20px",
    marginBottom: "10px",
  },
  message: {
    marginTop: "12px",
    marginBottom: "12px",
    color: tokens.colorPaletteRedForeground1,
    fontWeight: tokens.fontWeightMedium,
  },
});

const AnalyzeButton: React.FC<AnalyzeButtonProps> = (props: AnalyzeButtonProps) => {
  const [message, setMessage] = useState<string>("");
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const styles = useStyles();

  const handleAnalyze = async () => {
    try {
      setMessage("");
      setIsLoading(true);

      // Check API connection first using health endpoint
      try {
        await checkHealth();
      } catch {
        setMessage("Failed to connect to citation service. Please try again later.");
        return;
      }

      // Check document_id exists if a document_id is being passed to analyzeText
      if (DOCUMENT_ID) {
        try {
          const document = await getDocument();

          if (document.document_id !== DOCUMENT_ID) {
            setMessage("No information found on requested document_id.");
            return;
          }
        } catch (error) {
          setMessage("Unable to validate the requested document. Please try again.");
          return;
        }
```

```
      }

      // Get selected text
      const selectedText = await getSelectedText();

      // Handle no text selected
      if (!selectedText) {
        setMessage("Please select text before analyzing.");
        return;
      }

      // Analyze selected text
      const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

      await highlightSelectedText();

      const newCitation: CitationItem = {
        id: crypto.randomUUID(),
        selectedText: selectedText,
        sourceId: result.source_id,
        citationText: result.citation_text,
        confidence: result.confidence,
        url: result.url,
      };

      props.onCitationCreated(newCitation);
    } catch (error: any) {
      if (error.message === "ANALYZE_TIMEOUT") {
        setMessage("Citation analysis timed out. Please try again.");
      } else if (error.message === "NO_MATCHING_SOURCE") {
        setMessage("Unable to generate citation. Try refining selection.");
      } else {
        setMessage("Analyze request failed.");
      }
      console.error("Error analyzing text:", error);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={styles.analyzeContainer}>
      <Field className={styles.instructions}>Select text in the document, then click Analyze.</Field>

      {message && <div className={styles.message}>{message}</div>}

      <Button appearance="primary" size="large" onClick={handleAnalyze} disabled={isLoading}>
        {isLoading ? "Analyzing..." : "Analyze"}
      </Button>
    </div>
  );
};

export default AnalyzeButton;
```

What changed from your current file

Almost nothing structurally:

- imports stayed at the top
- constants stayed where they were
- **`useStyles`** stayed the same
- **`useState`** stayed the same
- **`handleAnalyze`** still runs in the same order:
    1. health check

2. document validation
3. get selected text
4. analyze selected text
5. highlight selected text

- the only new part after success is:

```
const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
};

props.onCitationCreated(newCitation);
```

What `App.tsx` needs now

Since this component now expects:

```
onCitationCreated: (citation: CitationItem) => void;
```

your **`App.tsx`** should pass that prop in.

At minimum, you'll need:

- a **`citations`** state array
- a handler that adds a new citation
- to render **`<AnalyzeButton onCitationCreated={handleCitationCreated} />`**

Minimal `CitationList.tsx`

Here's a simple scrolling pane component that matches this structure well:

```
import * as React from "react";
import { makeStyles, tokens } from "@fluentui/react-components";
import type { CitationItem } from "./AnalyzeText";

interface CitationListProps {
  citations: CitationItem[];
}

const useStyles = makeStyles({
  container: {
    marginTop: "30px",
    marginLeft: "20px",
    marginRight: "20px",
  },
  title: {
    fontWeight: tokens.fontWeightSemibold,
    marginBottom: "10px",
  },
  scrollPane: {
    maxHeight: "300px",
    overflowY: "auto",
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
  },
  emptyState: {
    color: tokens.colorNeutralForeground3,
  },
  citationCard: {
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
    marginBottom: "12px",
    backgroundColor: tokens.colorNeutralBackground2,
  },
  label: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "6px",
    marginBottom: "4px",
  },
  text: {
    wordBreak: "break-word",
  },
});

const CitationList: React.FC<CitationListProps> = (props: CitationListProps) => {
  const styles = useStyles();

  return (
    <div className={styles.container}>
      <div className={styles.title}>References</div>

      <div className={styles.scrollPane}>
        {props.citations.length === 0 ? (
          <div className={styles.emptyState}>No references yet.</div>
        ) : (
          props.citations.map((citation) => (
            <div key={citation.id} className={styles.citationCard}>
              <div className={styles.label}>Citation</div>
              <div className={styles.text}>{citation.citationText}</div>

              <div className={styles.label}>Source ID</div>
              <div className={styles.text}>{citation.sourceId}</div>

              <div className={styles.label}>Selected Text</div>
              <div className={styles.text}>{citation.selectedText}</div>
            </div>
          ))
        )}
      </div>
    </div>
  );
};

export default CitationList;
```

Minimal `App.tsx` change

This is the key part you'll need in **`App.tsx`**:

```
import * as React from "react";
import { useState } from "react";
import Header from "./Header";
import HeroList, { HeroListItem } from "./HeroList";
import TextInsertion from "./TextInsertion";
import AnalyzeButton, { CitationItem } from "./AnalyzeText";
import CitationList from "./CitationList";
import { makeStyles } from "@fluentui/react-components";
import { Ribbon24Regular, LockOpen24Regular, DesignIdeas24Regular } from "@fluentui/react-icons";
import { insertText } from "../taskpane";

interface AppProps {
  title: string;
}

const useStyles = makeStyles({
  root: {
    minHeight: "100vh",
  },
});

const App: React.FC<AppProps> = (props: AppProps) => {
  const styles = useStyles();
  const [citations, setCitations] = useState<CitationItem[]>([]);

  const listItems: HeroListItem[] = [
    {
      icon: <Ribbon24Regular />,
      primaryText: "Achieve more with Office integration",
    },
    {
      icon: <LockOpen24Regular />,
      primaryText: "Unlock features and functionality",
    },
    {
      icon: <DesignIdeas24Regular />,
      primaryText: "Create and visualize like a pro",
    },
  ];

  const handleCitationCreated = (citation: CitationItem) => {
    setCitations((prevCitations) => [citation, ...prevCitations]);
  };

  return (
    <div className={styles.root}>
      <Header logo="assets/logo-filled.png" title={props.title} message="Welcome" />
      <HeroList message="Discover what this add-in can do for you today!" items={listItems} />
      <TextInsertion insertText={insertText} />
      <AnalyzeButton onCitationCreated={handleCitationCreated} />
      <CitationList citations={citations} />
    </div>
  );
};

export default App;
```

Why this keeps the mapping you need

Each citation now stores:

`selectedText`

So every reference in the pane is linked to the exact text that was highlighted when the citation was created.

That gives you a clean session-level mapping:

- highlighted text in Word
- corresponding citation object in the pane

The next step after this is making each citation clickable and using `selectedText` to find and re-select the matching text in the document.

yes let's move on to the next step

Great — the next step is to make each citation in the References pane clickable so it re-selects the linked text in Word.

Since you're already storing `selectedText` for each citation, the simplest MVP approach is:

1. user clicks a citation card
2. search the Word document for that citation's `selectedText`
3. select the first matching result in the document

That fits your current session-only design well.

What to add

You need two pieces:

- a helper function that finds text in the Word document and selects it
- an `onClick` handler in `CitationList.tsx`

---

1. Add a select helper

Since you've been keeping Word-related helpers in `taskpane.ts`, add this there.

```
export async function selectTextInDocument(textToFind: string): Promise<boolean> {
  return Word.run(async (context) => {
    const searchResults = context.document.body.search(textToFind, {
      matchCase: false,
      matchWholeWord: false,
    });

    searchResults.load("items");

    await context.sync();

    if (searchResults.items.length === 0) {
      return false;
    }

    searchResults.items[0].select();
    await context.sync();

    return true;
  });
}
```

What it does

- searches the document body for the selected text
- loads the search results
- if at least one match exists, selects the first one
- returns **`true`** if successful, **`false`** otherwise

---

2. Update `CitationList.tsx`

Now make each citation card clickable.

Updated `CitationList.tsx`

```
import * as React from "react";
import { makeStyles, tokens } from "@fluentui/react-components";
import type { CitationItem } from "./AnalyzeText";
import { selectTextInDocument } from "../taskpane";

interface CitationListProps {
  citations: CitationItem[];
}

const useStyles = makeStyles({
  container: {
    marginTop: "30px",
    marginLeft: "20px",
    marginRight: "20px",
  },
  title: {
    fontWeight: tokens.fontWeightSemibold,
    marginBottom: "10px",
  },
  scrollPane: {
    maxHeight: "300px",
    overflowY: "auto",
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
  },
  emptyState: {
    color: tokens.colorNeutralForeground3,
  },
  citationCard: {
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
    marginBottom: "12px",
    backgroundColor: tokens.colorNeutralBackground2,
    cursor: "pointer",
  },
  label: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "6px",
    marginBottom: "4px",
  },
  text: {
    wordBreak: "break-word",
  },
  helperText: {
    fontSize: tokens.fontSizeBase200,
    color: tokens.colorNeutralForeground3,
    marginBottom: "10px",
  },
});

const CitationList: React.FC<CitationListProps> = (props: CitationListProps) => {
  const styles = useStyles();

  const handleCitationClick = async (selectedText: string) => {
    try {
      const found = await selectTextInDocument(selectedText);

      if (!found) {
        console.log("Could not find linked text in the document.");
      }
    } catch (error) {
      console.error("Error selecting citation text:", error);
    }
  };

  return (
    <div className={styles.container}>
      <div className={styles.title}>References</div>
      <div className={styles.helperText}>Click a reference to jump to its linked text.</div>

      <div className={styles.scrollPane}>
        {props.citations.length === 0 ? (
          <div className={styles.emptyState}>No references yet.</div>
```

```
        ) : (
          props.citations.map((citation) => (
            <div
              key={citation.id}
              className={styles.citationCard}
              onClick={() => handleCitationClick(citation.selectedText)}
            >
              <div className={styles.label}>Citation</div>
              <div className={styles.text}>{citation.citationText}</div>

              <div className={styles.label}>Source ID</div>
              <div className={styles.text}>{citation.sourceId}</div>

              <div className={styles.label}>Selected Text</div>
              <div className={style.text}>{citation.selectedText}</div>
            </div>
          ))
        )}
      </div>
    </div>
  );
};

export default CitationList;
```

---

3. Why this works with your current mapping

You already store:

`selectedText`

inside each citation object.

That means the citation pane already knows exactly what text to search for when clicked. So you do not need any major refactor.

The mapping is now:

- highlighted text in Word
- stored **`selectedText`** in citation object
- click citation
- search document for **`selectedText`**
- select first match

---

4. One limitation to be aware of

This MVP approach works best when the selected text is fairly unique.

If the same text appears multiple times in the document, **`search()`** will select the first match, not necessarily the exact original location.

That is acceptable for MVP and interview-demo purposes, especially because your design is intentionally session-only and lightweight.

A good way to explain it is:

> “For the MVP, I used selected text as the session mapping key. Clicking a citation searches the document and re-selects the first match. A more advanced version could store richer range

metadata for more precise navigation."

That sounds thoughtful and practical.

---

5. Optional user-facing error message

Right now, if the text is not found, it only logs to console:

```
console.log("Could not find linked text in the document.");
```

That's okay for a first pass, but you may want to show a visible message in the pane later.

For now, keep it simple unless you want a fully polished UX.

---

6. What to test

Test 1

Analyze one passage, then click its citation.

Expected:

- Word jumps to and selects that text

Test 2

Analyze two different passages, then click each citation.

Expected:

- each one selects the matching text

Test 3

If the document text changes after analysis and the exact text is no longer present:

Expected:

- nothing breaks
- it just fails to find the text

---

7. At this point, your MVP flow is basically complete

You now have:

- select text
- analyze selected text
- validate health/document
- handle timeout/low-confidence/no-match

- highlight selected text
- show citations in a scrollable pane
- click citation to jump back to linked text

That is the core end-to-end product.

Best next move

The next thing I'd do is a polish pass:

- remove default sample UI like `HeroList` and maybe `TextInsertion`
- improve citation card styling
- optionally show confidence and URL
- make error messages a bit more polished for demo day

Paste your current `CitationList.tsx` if you want me to adapt this exactly to your file.

> is there an alternative to searching for the same text in order to link it?

Yes — there are a few alternatives, but each has tradeoffs.

For a Word add-in MVP, the main options are:

1. Search by the selected text

This is what you have now.

Pros

- simple
- easy to implement
- works with session-only storage

Cons

- if the same text appears multiple times, it may jump to the wrong one
- if the text changes later, the link can break

---

2. Store a richer text snapshot

Instead of storing only `selectedText`, store:

- `selectedText`
- a short prefix before it

- a short suffix after it

So each citation keeps a small text “fingerprint,” like:

```
{
  selectedText: "trust and safety reviews are required...",
  prefix: "According to internal policy, ",
  suffix: " before production deployment."
}
```

Then when the user clicks the citation, you search for the selected text and use the surrounding context to disambiguate the correct match.

Pros

- much more reliable than plain text search
- still session-only
- still relatively easy

Cons

- a bit more logic to implement
- still not perfect if the document is edited heavily

This is probably the **best alternative** if you want something better than plain text search without making the architecture too complex.

---

3. Store the selection range immediately after analyze

Instead of re-searching later, you can try to store information tied to the original selected range.

For example, when the user selects text and clicks Analyze, you can grab the selection range and try to keep a reference to it in memory for the session.

Pros

- closer to the exact original location
- avoids re-searching by text

Cons

- Office.js range objects are not ideal to keep directly in React state
- they are tied to a `Word.run` context
- they can become unreliable across async boundaries and later interactions

So this sounds good in theory, but in practice it is usually the trickiest option.

---

4. Insert a hidden marker / content control / comment

You can attach something to the document itself at the selected location, then link your citation to that marker.

Examples:

- insert a content control around the selected text
- insert a comment
- insert hidden metadata if supported by your design

Then clicking the citation could target that inserted object instead of searching text.

Pros

- strongest link to the exact location
- survives repeated interactions better

Cons

- modifies the document structure
- adds complexity
- may be too heavy for your MVP
- your project seems to prefer minimal mutation beyond highlighting

This is more like a stretch feature than the best MVP approach.

---

5. Store an ordinal position or nearby paragraph index

At analyze time, capture some structural location info, like:

- paragraph text
- paragraph index
- selection offset within paragraph

Then later, navigate back using that structure.

Pros

- more precise than plain search
- can work well if the document structure is stable

Cons

- harder to implement
- more brittle if the document changes
- depends on how much paragraph/location detail Office.js gives you cleanly

---

Best recommendation for your project

For your interview project, I would choose one of these two:

Best simple option

Search using selected text

- fastest
- easiest to explain
- enough for MVP

Best improved option

Store selected text + surrounding context

- still lightweight
- noticeably more reliable
- easy to describe as a thoughtful improvement

If you want to sound strong in the presentation, you can say:

> For the MVP, I linked citations back to text using session-only text matching. A stronger next iteration would store surrounding context with the selection to make navigation more precise when the same text appears multiple times.

That shows product judgment and engineering awareness.

What I would not recommend right now

I would avoid:

- storing raw Office range objects in React state
- adding content controls/comments just for linking

Those are valid ideas, but they add more complexity than you probably need in a 24-hour project.

Practical upgraded design

A very good middle ground is to store:

```
interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
  selectionPreviewStart?: string;
  selectionPreviewEnd?: string;
}
```

Then later when the citation is clicked, you can:

1. search for `selectedText`

2. if multiple results come back, compare nearby context
3. pick the closest match

That gives you a stronger link without changing your current architecture much.

If you want, I can show you how to implement the **selected text + context** version while keeping your current files mostly unchanged.

> Ok here's my idea. User selects text and clicks analyze. If citation is generated, the selected text should be highlighted. At the end of that text, it should insert the in-text citation (will need to parse it from the citation returned from the response using regex around the punctuation). Add a comment to the in-text citation that was just inserted. Create a direct link between the references list and the in-text citation through the comment. The comment can include only the confidence score. That way, when a user clicks on a citation from the citation list, it'll direct them to the comment.

Yes — that design is workable, and it's stronger than plain text search because you'd be linking each reference to a concrete object in the document instead of re-finding the same text later.

The parts of your plan that are directly supported by the Word JavaScript API are:

- permanently highlighting the selected range via the range font highlight color,
- inserting a comment on a range with `insertComment(...)`,
- getting all comments in the document body with `body.getComments()`,
- and getting the document range associated with a comment via `comment.getRange()`. Word also supports inserting bookmarks on a range if you want a backup anchor strategy. Microsoft Learn+3

Here's the architecture I'd use.

Recommended flow

1. User selects text.
2. You call `/analyze`.
3. If analyze succeeds:
    - highlight the selected text,
    - compute and insert the in-text citation at the end of that selected text,
    - insert a comment on the newly inserted in-text citation,
    - store a reference entry in your React state containing:
        - `citationText`
        - `selectedText`
        - `commentId`
        - `sourceId`

    - `confidence`
    - `url`
4. When the user clicks a reference in the References pane:
    - load comments from `context.document.body.getComments()`,
    - find the comment whose `id` matches the stored `commentId`,
    - call `comment.getRange().select()` to jump to that in-text citation area. `Comment.id` and `Comment.getRange()` are both part of the comment model, and `body.getComments()` is the safer collection entry point for a Word-on-the-web target than `document.comments`, which is listed as desktop-only. Microsoft Learn+3

That means your "link" would really be:
reference list item → stored `commentId` → matching `Comment` object → `comment.getRange()` → select that citation in the document. The API clearly supports getting a comment's range; that is the most reliable navigation target in your proposed design. Microsoft Learn+1

There is one part I would tweak slightly: I would not make the comment text itself the primary identifier. I would make the **comment ID** the primary identifier, and put only the confidence score in the comment body, exactly like you suggested. `Comment` objects expose an `id`, and comment content is plain text, so `"Confidence: 0.95"` is a good payload for the comment body while the stored `commentId` remains your real mapping key. Microsoft Learn+1

A good citation state shape would be:

```
interface CitationItem {
  id: string;              // local React id
  selectedText: string;
  sourceId: string;
  citationText: string;    // full citation from backend
  inTextCitation: string;  // parsed short form inserted into doc
  commentId: string;       // Word comment id
  confidence?: number;
  url?: string;
}
```

That gives you two separate concerns:

- `citationText` for the References pane,
- `commentId` for navigation.

For the actual document mutation, I'd do it in this order:

1. capture the selected range,
2. highlight it,
3. insert the in-text citation **after the selected text**,
4. keep the returned inserted range,
5. call `insertComment` on that inserted citation range,
6. load the created comment's `id`,
7. add the citation item to state.

That sequence is important because the comment should be attached to the in-text citation, not the original selected text. `insertComment(...)` is a range method in WordApi 1.4, so the cleanest implementation is to comment the range returned from inserting the in-text citation. Microsoft Learn+1

For parsing the in-text citation, your regex idea is fine. I'd keep it simple:

- if the returned citation looks like `Author, Title, 2025, https://...`, extract a short form like `(Author, 2025)`,
- if it doesn't match your expected pattern, fall back to a generic `(Source)` or the first meaningful clause.
  That parsing is app-specific logic, so it's okay to keep it heuristic for the MVP.

The main implementation risk is not the comment itself — it's **how you later look that comment up**. The API supports getting comments from the body and each comment has an `id` and a document range. So your click handler should not search text; it should:

- call `body.getComments()`,
- load the items and their `id`s,
- find the matching one,
- call `getRange().select()`. `Body.getComments()` is in WordApi 1.4, and `Comment.getRange()` is also documented in WordApi 1.4. Microsoft Learn+2

One important practical note: for a **Word on the web** MVP, use `body.getComments()` rather than `document.comments`. Microsoft's docs show `body.getComments()` in WordApi 1.4, while `document.comments` is marked under `WordApiDesktop 1.4`, so the body-based approach is the safer one for your stated platform target. Microsoft Learn+1

If you want a fallback in case comment navigation ends up awkward in the web client, the best backup anchor is a **bookmark** inserted on the in-text citation range. Word supports `range.insertBookmark(name)` and also `getBookmarks(...)` on ranges. So the robust version is:

- comment for user-visible metadata,
- bookmark for internal navigation anchor. Microsoft Learn+1

So my recommendation is:

Best version

- highlight original selected text,
- insert in-text citation after it,
- insert comment on the in-text citation,
- store `commentId`,
- click reference → find comment by `commentId` via `body.getComments()` → `comment.getRange().select()`.

Most robust version

- do all of the above,
- also insert a bookmark on the same citation range,

- store both `commentId` and `bookmarkName`.

That gives you a very clean story in the interview:

- the selected source text is visibly highlighted,
- the generated citation is inserted into the document,
- the confidence is attached as comment metadata,
- the References pane is linked directly to the document through a stable document object instead of text search.

The next concrete step is to refactor your success path so it returns both the inserted citation range and the new comment ID.

> ok awesome. Let's start by parsing the in-text citation. The api gives me the citation in the response in APA format which starts with {Author}. {Year}. {Title}. {url} Essentially we should use the periods to extract the Author and year into variables, then the in-text citation will be in the form ({Author}, {Year})

Good starting step. Since the API always returns citations in the format:

```
Author. Year. Title. url
```

you can reliably parse the **Author** and **Year** using the periods as delimiters.

The safest approach is:

1. Split the citation string on `"."`
2. Trim whitespace
3. Extract the first two segments
4. Format the in-text citation as `(Author, Year)`

This keeps the logic simple and predictable.

Helper Function: Parse In-Text Citation

Create a helper function (either inside `AnalyzeText.tsx` or in a small `utils/citation.ts` file).

```
function parseInTextCitation(citation: string): { author: string; year: string; inText: string } {
  const parts = citation.split(".");

  const author = parts[0]?.trim() || "Unknown";
  const year = parts[1]?.trim() || "n.d.";

  const inText = `(${author}, ${year})`;

  return { author, year, inText };
}
```

Example

Input:

```
Smith. 2023. Trust and Safety Handbook. https://example.com
```

Output:

```
author: "Smith"
year: "2023"
inText: "(Smith, 2023)"
```

How to Use It in `handleAnalyze`

Right after the analyze request succeeds:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

const { author, year, inText } = parseInTextCitation(result.citation_text);
```

Now you have:

```
author = "Smith"
year = "2023"
inText = "(Smith, 2023)"
```

Example Integration in Your Existing Flow

Replace your success block with:

```
// Analyze selected text
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

// Parse citation
const { author, year, inText } = parseInTextCitation(result.citation_text);

await highlightSelectedText();

console.log("Author:", author);
console.log("Year:", year);
console.log("In-text citation:", inText);
```

Later this **`inText`** variable will be used for:

1. **Inserting the citation in the document**
2. **Adding the comment to that citation**
3. **Linking the citation list entry**

Edge Case Protection (Recommended)

If the API response ever changes format, the parser should fail safely.

Updated version:

```
function parseInTextCitation(citation: string) {
  if (!citation) {
    return {
      author: "Unknown",
      year: "n.d.",
      inText: "(Unknown, n.d.)"
    };
  }

  const parts = citation.split(".");

  const author = parts[0]?.trim() || "Unknown";
  const year = parts[1]?.trim() || "n.d.";

  return {
    author,
    year,
    inText: `(${author}, ${year})`
  };
}
```

Test Examples

Example 1

```
Johnson. 2021. Trust Systems Overview. https://example.com
```

Result

```
(Johnson, 2021)
```

Example 2

```
TrustOps Team. 2025. Policy Handbook. https://example.com
```

Result

```
(TrustOps Team, 2025)
```

Next Step

Once this parser is working, the **next step** is:

Insert the in-text citation **immediately after the selected text**.

The sequence will become:

```
1. user selects text
2. analyze
3. parse citation
4. highlight selected text
5. insert "(Author, Year)" after the selection
6. attach comment to that inserted citation
7. store commentId for linking
```

That step requires only about **15 lines of Office.js code**, and it will give you the core behavior of your linking system.

ok let's get the insert text after end of selection functionality implemented

Yes — the clean way to do this is to insert the parsed in-text citation **after the current selection range**. The Word JavaScript API supports inserting text at a range with `insertText(..., Word.InsertLocation.after)`. Microsoft Learn+1

Add this helper

Since you've been keeping document operations together, add this function where your Word helpers live:

```
export async function insertCitationAfterSelection(inTextCitation: string): Promise<void> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);

    await context.sync();
  });
}
```

This inserts a space plus the citation, so:

```
Selected sentence here.
```

becomes:

```
Selected sentence here. (Smith, 2023)
```

If you want the inserted range back

You'll probably want this soon for adding the comment to the citation itself. In that case, use this version instead:

```
export async function insertCitationAfterSelection(inTextCitation: string): Promise<Word.Range> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);
    insertedRange.load("text");

    await context.sync();

    return insertedRange;
  });
}
```

That uses the returned `Range` from `insertText`, which is useful for the next step. The `Range` API documents `insertText(text, insertLocation)`. Microsoft Learn

Then in your analyze flow

After parsing the APA citation and after analyze succeeds, do:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

const { inText } = parseInTextCitation(result.citation_text);

await highlightSelectedText();
await insertCitationAfterSelection(inText);
```

Example integration in your current order

Inside **`handleAnalyze`**, the success section becomes:

```
// Analyze selected text
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

// Parse in-text citation
const { inText } = parseInTextCitation(result.citation_text);

// Highlight selected text
await highlightSelectedText();

// Insert in-text citation after selection
await insertCitationAfterSelection(inText);
```

Recommended parser

```
function parseInTextCitation(citation: string): { author: string; year: string; inText: string } {
  const parts = citation.split(".");

  const author = parts[0]?.trim() || "Unknown";
  const year = parts[1]?.trim() || "n.d.";

  return {
    author,
    year,
    inText: `(${author}, ${year})`,
  };
}
```

One important note

This will insert the citation after the currently selected range. If the user changes the selection while the analyze request is in flight, the insertion point could shift. For your MVP, this is acceptable. A more robust version would capture and reuse the original range, but this is the fastest solid implementation.

The next step after this is to return the inserted citation range and attach the comment directly to that range.

> okay so actually in the parsing function, we need to remove the parenthesis that wrap around the year being passed back from the citation response

Good catch — APA references often return the year like:

```
Smith. (2023). Trust and Safety Handbook. https://example.com
```

If you split on `"."`, the year part becomes:

```
"(2023)"
```

So you need to **strip the parentheses** before constructing the in-text citation.

The cleanest way is to remove `(` and `)` using a small regex.

Updated Parsing Function

```
function parseInTextCitation(citation: string): { author: string; year: string; inText: string } {
  if (!citation) {
    return {
      author: "Unknown",
      year: "n.d.",
      inText: "(Unknown, n.d.)"
    };
  }

  const parts = citation.split(".");

  const author = parts[0]?.trim() || "Unknown";

  // Remove parentheses around year
  const rawYear = parts[1]?.trim() || "n.d.";
  const year = rawYear.replace(/[()]/g, "");

  const inText = `(${author}, ${year})`;

  return { author, year, inText };
}
```

Example

API response

```
Smith. (2023). Trust and Safety Handbook. https://example.com
```

Parsing result

```
author = "Smith"
year = "2023"
inText = "(Smith, 2023)"
```

Why regex is better here

Using:

```
.replace(/[()]/g, "")
```

removes both parentheses safely:

Input	Output
`(2023)`	`2023`
`(1998)`	`1998`
`2023`	`2023`

So it works whether the API includes parentheses or not.

Optional improvement (more robust parsing)

If you want to make it slightly more defensive:

```
const rawYear = parts[1]?.trim() || "";
const yearMatch = rawYear.match(/\d{4}/);
const year = yearMatch ? yearMatch[0] : "n.d.";
```

This extracts the **first 4-digit year**, which protects against formats like:

```
(2023).
2023.
2023a.
```

But your regex removal approach is already perfectly fine for your expected response format.

Your flow now becomes

```
1. User selects text
2. Analyze request
3. Parse citation
4. Highlight selected text
5. Insert "(Author, Year)" after selection
6. Attach comment to inserted citation
7. Store commentId for reference linking
```

If you'd like, the **next step (attaching the comment to the inserted citation)** is where the architecture becomes really elegant, because the **comment ID becomes your permanent link between the References pane and the document**.

> ok next is attaching the comment to the in-text citation

Yes — the next step is to attach the comment to the **inserted in-text citation range**, not the original selected text. Word's `Range` API supports both `insertText(...)` and `insertComment(...)`, so the clean pattern is:

1. get the current selection
2. insert the in-text citation after it
3. capture the returned range for that inserted text
4. call `insertComment(...)` on that returned range
5. load the new comment's `id` so you can store it in your citation item for navigation later. `insertComment` is supported on `Word.Range`, and comments were added in WordApi 1.4. Microsoft Learn+1

Best helper to add

Since you want the comment to contain only confidence, make one helper that both:

- inserts the in-text citation
- adds the comment
- returns the comment id

In your Word/helper file

```
export async function insertCitationAndComment(
  inTextCitation: string,
  confidence?: number
): Promise<{ commentId: string; insertedText: string }> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);

    const commentText =
      confidence !== undefined ? `Confidence: ${confidence}` : "Confidence: unavailable";

    const comment = insertedRange.insertComment(commentText);
    comment.load("id");

    await context.sync();

    return {
      commentId: comment.id,
      insertedText: insertedRange.text,
    };
  });
}
```

Why this is the right pattern

`insertText(..., Word.InsertLocation.after)` inserts text immediately after the selected range and returns a **`Range`**. Then **`insertComment(...)`** can be called on that range directly. That gives you a concrete document anchor for later navigation. Microsoft Learn+1

Then in `AnalyzeText.tsx`

After analyze succeeds and after you parse the in-text citation, replace:

```
await highlightSelectedText();
await insertCitationAfterSelection(inText);
```

with:

```
await highlightSelectedText();

const { commentId } = await insertCitationAndComment(inText, result.confidence);
```

Then include **`commentId`** in the citation object you store:

```
const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
  commentId: commentId,
};
```

Update your citation type

Add **`commentId`**:

```
export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
  commentId: string;
}
```

Full helper flow

Your successful analyze path now becomes:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

const { inText } = parseInTextCitation(result.citation_text);

await highlightSelectedText();

const { commentId } = await insertCitationAndComment(inText, result.confidence);

const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
  commentId: commentId,
};

props.onCitationCreated(newCitation);
```

One small improvement

If you want the visible inserted text and the comment target to be guaranteed identical, you can also **`load("text")`** on **`insertedRange`** before **`context.sync()`**, but it is not required unless you need that text back in code. The core needed property is the comment **`id`**. Microsoft Learn

Important note for Word on the web

Comments are part of WordApi 1.4. Since your project is targeting Word on the web, this is a good fit, but make sure your manifest/support assumptions align with that requirement set. The comments feature was added in WordApi 1.4. Microsoft Learn+1

What to test

Test one success case and confirm:

- selected text gets highlighted
- **`(Author, Year)`** is inserted after the selection
- a comment appears on that inserted in-text citation
- your citation item now stores a real **`commentId`**

The next step after this is the click behavior: when the user clicks a citation in the references pane, use the stored **`commentId`** to find that comment and select its range.

Ok I've got the comment add-in working correctly but now my references no longer show up in my task pane and I get an error message "Analyze request failed"

Most likely, the **analyze request itself is succeeding**, but something **after** it is throwing inside **`handleAnalyze`**, so your catch block falls through to:

```
setMessage("Analyze request failed.");
```

That would explain both symptoms at once:

- the document mutation works
- the reference never gets added to state
- you only see the generic error message

The most likely failure point is in the new **insert citation + comment** logic or in building the new citation object afterward.

First thing to do

Temporarily make your catch block show the actual error message:

```
} catch (error: any) {
  console.error("Full error analyzing text:", error);

  if (error.message === "ANALYZE_TIMEOUT") {
    setMessage("Citation analysis timed out. Please try again.");
  } else if (error.message === "NO_MATCHING_SOURCE") {
    setMessage("Unable to generate citation. Try refining selection.");
  } else {
    setMessage(`Analyze request failed: ${error.message}`);
  }
}
```

That will tell you whether the problem is:

- comment insertion
- return shape
- missing loaded property
- citation object creation
- prop callback issue

Most likely cause

If your helper looks anything like this:

```
export async function insertCitationAndComment(
  inTextCitation: string,
  confidence?: number
): Promise<{ commentId: string; insertedText: string }> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);

    const commentText =
      confidence !== undefined ? `Confidence: ${confidence}` : "Confidence: unavailable";

    const comment = insertedRange.insertComment(commentText);
    comment.load("id");

    await context.sync();

    return {
      commentId: comment.id,
      insertedText: insertedRange.text,
    };
  });
}
```

then the issue is probably this line:

```
insertedText: insertedRange.text
```

because **`insertedRange.text`** was **never loaded**.

In Office.js, if you access a property that wasn't loaded, you can get an **`OfficeExtension.Error`** even though the actual document action already happened.

That would fit your symptoms perfectly:

- citation inserted
- comment inserted
- then code crashes when returning **`insertedRange.text`**
- **`onCitationCreated(...)`** never runs
- task pane never updates

Fix option 1 — simplest

If you do **not** need **`insertedText`**, remove it entirely:

```
export async function insertCitationAndComment(
  inTextCitation: string,
  confidence?: number
): Promise<{ commentId: string }> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);

    const commentText =
      confidence !== undefined ? `Confidence: ${confidence}` : "Confidence: unavailable";

    const comment = insertedRange.insertComment(commentText);
    comment.load("id");

    await context.sync();

    return {
      commentId: comment.id,
    };
  });
}
```

Then in **`AnalyzeText.tsx`**:

```
const { commentId } = await insertCitationAndComment(inText, result.confidence);
```

That is the cleanest fix.

Fix option 2 — if you do want `insertedText`

Then you must load it first:

```
export async function insertCitationAndComment(
  inTextCitation: string,
  confidence?: number
): Promise<{ commentId: string; insertedText: string }> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);
    insertedRange.load("text");

    const commentText =
      confidence !== undefined ? `Confidence: ${confidence}` : "Confidence: unavailable";

    const comment = insertedRange.insertComment(commentText);
    comment.load("id");

    await context.sync();

    return {
      commentId: comment.id,
      insertedText: insertedRange.text,
    };
  });
}
```

Second likely cause to check

Make sure your citation object matches your current interface.

For example, if your interface now requires:

```
commentId: string;
```

but your new citation object does not include it, TypeScript may complain at build time, or runtime code may behave unexpectedly depending on how things are set up.

Your **`newCitation`** should look like:

```
const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
  commentId: commentId,
};
```

Third likely cause to check

Make sure **`props.onCitationCreated(newCitation);`** is still being called **after** the comment insertion succeeds.

Your success section should be in this order:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

const { inText } = parseInTextCitation(result.citation_text);

await highlightSelectedText();

const { commentId } = await insertCitationAndComment(inText, result.confidence);

const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
  commentId: commentId,
};

props.onCitationCreated(newCitation);
```

If the error happens before that last line, references won't show.

Best debugging move

Add logs between each step so you can find the exact line that breaks:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);
console.log("analyzeText success", result);

const { inText } = parseInTextCitation(result.citation_text);
console.log("parsed in-text citation", inText);

await highlightSelectedText();
console.log("highlight success");

const { commentId } = await insertCitationAndComment(inText, result.confidence);
console.log("comment success", commentId);

const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
  commentId: commentId,
};
console.log("citation object created", newCitation);

props.onCitationCreated(newCitation);
console.log("citation added to task pane state");
```

That will tell you exactly where it stops.

My strongest guess

The issue is **accessing an unloaded Office.js property**, most likely `insertedRange.text`.

So the fix I'd try first is:

In your helper, change this:

```
return {
  commentId: comment.id,
  insertedText: insertedRange.text,
};
```

to this:

```
return {
  commentId: comment.id,
};
```

and adjust the return type accordingly.

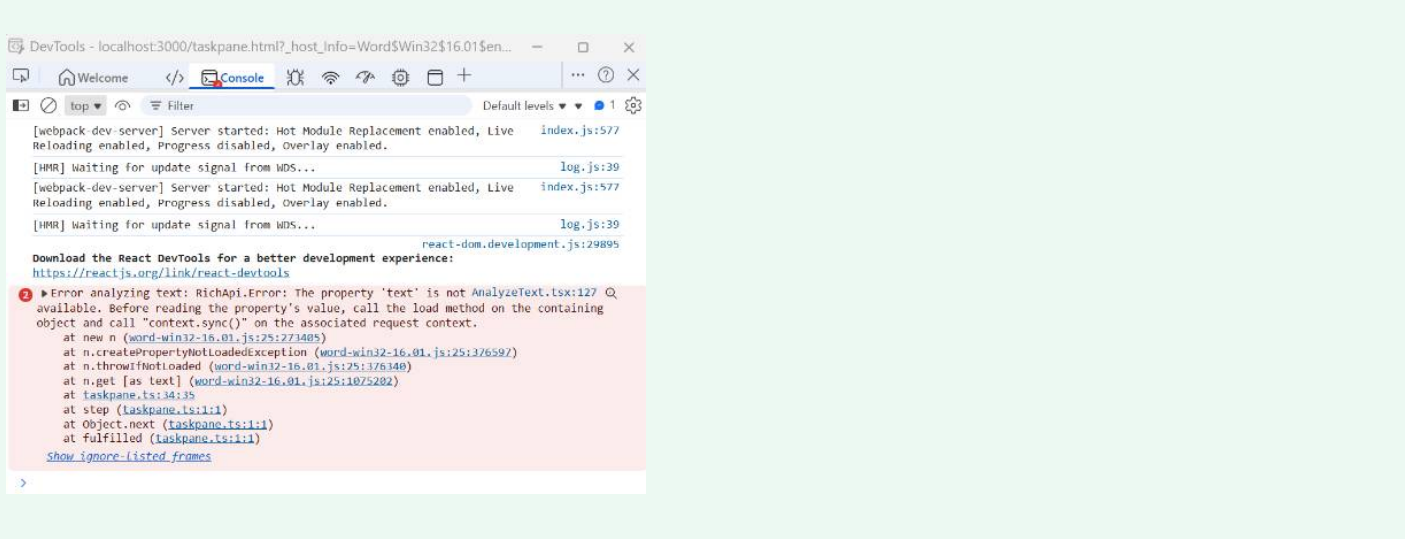
After that, test again

If that fixes it, your references should start appearing again.

If not, paste these two pieces:

- your current `insertCitationAndComment()` helper
- your current success block in `AnalyzeText.tsx`

and I'll pinpoint the exact failing line.



The error is clear:

> The property **`text`** is not available. Before reading the property's value, call the load method on the containing object and call **`context.sync()`**.

So somewhere in your Office.js code, you are trying to read **`.text`** from a **`Range`** that was never loaded.

The console also points to:

- **`AnalyzeText.tsx:127`**
- **`taskpane.ts:34:35`**

That matches exactly the issue we suspected: you are likely returning **`insertedRange.text`** without doing:

```
insertedRange.load("text");
await context.sync();
```

What's happening

Office.js objects are lazy-loaded. You cannot do this:

```
const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);

return {
  commentId: comment.id,
  insertedText: insertedRange.text,
};
```

because **`insertedRange.text`** has not been loaded yet.

Even though the insertion and comment may succeed in the document, the code crashes when it tries to read that unloaded property. Then your **`catch`** runs, which is why you see:

```
Analyze request failed.
```

and your reference never gets added to the task pane.

Fix

Best fix: stop returning `insertedText` if you do not need it

Update your helper so it only returns **`commentId`**.

In `taskpane.ts` or wherever that helper lives

Change it to:

```
export async function insertCitationAndComment(
  inTextCitation: string,
  confidence?: number
): Promise<{ commentId: string }> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);

    const commentText =
      confidence !== undefined ? `Confidence: ${confidence}` : "Confidence: unavailable";

    const comment = insertedRange.insertComment(commentText);
    comment.load("id");

    await context.sync();

    return {
      commentId: comment.id,
    };
  });
}
```

Then in **`AnalyzeText.tsx`**, keep:

```
const { commentId } = await insertCitationAndComment(inText, result.confidence);
```

That should solve it.

If you do want to keep `insertedText`

Then you must load it first:

```
export async function insertCitationAndComment(
  inTextCitation: string,
  confidence?: number
): Promise<{ commentId: string; insertedText: string }> {
  return Word.run(async (context) => {
    const selection = context.document.getSelection();

    const insertedRange = selection.insertText(` ${inTextCitation}`, Word.InsertLocation.after);
    insertedRange.load("text");

    const commentText =
      confidence !== undefined ? `Confidence: ${confidence}` : "Confidence: unavailable";

    const comment = insertedRange.insertComment(commentText);
    comment.load("id");

    await context.sync();

    return {
      commentId: comment.id,
      insertedText: insertedRange.text,
    };
  });
}
```

But unless you actually use **`insertedText`**, I would remove it.

Why your references disappeared

Because the code likely crashes here before it reaches:

```
props.onCitationCreated(newCitation);
```

So this sequence is happening:

1. analyze succeeds
2. highlight succeeds
3. comment gets inserted
4. helper tries to read **`insertedRange.text`**
5. Office.js throws
6. catch block runs
7. no citation gets added to React state
8. task pane stays empty

What to do right now

Go to the helper in **`taskpane.ts`** around line 34 and look for anything like:

```
insertedRange.text
```

If you see that, remove it or load it properly.

After fixing it

Your success flow should work again:

- selected text highlighted
- in-text citation inserted
- comment attached
- reference added to task pane

Recommended safer version of the success block

Once the helper is fixed, your success logic should be:

```
const result = await analyzeText(selectedText, DOCUMENT_ID, USER_ID);

const { inText } = parseInTextCitation(result.citation_text);

await highlightSelectedText();

const { commentId } = await insertCitationAndComment(inText, result.confidence);

const newCitation: CitationItem = {
  id: crypto.randomUUID(),
  selectedText: selectedText,
  sourceId: result.source_id,
  citationText: result.citation_text,
  confidence: result.confidence,
  url: result.url,
  commentId: commentId,
};

props.onCitationCreated(newCitation);
```

One more debugging trick

While testing, temporarily change your generic error message to:

```
setMessage(`Analyze request failed: ${error.message}`);
```

That will save you time on the next bug.

Paste your current **`insertCitationAndComment()`** helper if you want me to check the exact line.

> Awesome that's all implemented. Now, is there a way to make it so that when I click on my citation from my reference pane it selects either the associated text with the reference or the comment that I inserted over the in-text citation? Think of it as a click-to-jump mechanism that will take me to my reference location in the document

Yes. The best click-to-jump mechanism in your current design is to use the **stored `commentId`** and jump to the **range of that Word comment**, instead of searching the text again. Word's API supports:

- getting all comments from the document body with **`body.getComments()`**,
- reading each comment's **`id`**,
- and getting the document range a comment is attached to with **`comment.getRange()`**. That range can then be selected. These comment APIs are part of WordApi 1.4, which is the right requirement set for your comment-based design. Microsoft Learn+2

So your mapping should be:

```
reference pane item -> stored commentId -> matching Word comment -> comment.getRange() -> select()
```

That is stronger than text search because it links the reference to an actual document object you created when you inserted the comment. Microsoft Learn+1

What to implement

You already store **`commentId`** in each citation item. Now add a helper that:

1. gets all comments from the document body
2. loads their ids
3. finds the one matching your stored **`commentId`**
4. gets that comment's range
5. selects it

Add this helper

Put this with your Word helpers:

```
export async function selectCommentById(commentId: string): Promise<boolean> {
  return Word.run(async (context) => {
    const comments = context.document.body.getComments();
    comments.load("items/id");

    await context.sync();

    const targetComment = comments.items.find((comment) => comment.id === commentId);

    if (!targetComment) {
      return false;
    }

    const commentRange = targetComment.getRange();
    commentRange.select();

    await context.sync();

    return true;
  });
}
```

This works because **`Body.getComments()`** returns the comments associated with the body, and **`Comment.getRange()`** returns the range in the main document where the comment is attached.

Microsoft Learn+1

Then update your `CitationList.tsx`

Make each citation clickable and use **`commentId`** instead of **`selectedText`**.

```
import * as React from "react";
import { makeStyles, tokens } from "@fluentui/react-components";
import type { CitationItem } from "./AnalyzeText";
import { selectCommentById } from "../taskpane";

interface CitationListProps {
  citations: CitationItem[];
}

const useStyles = makeStyles({
  container: {
    marginTop: "30px",
    marginLeft: "20px",
    marginRight: "20px",
  },
  title: {
    fontWeight: tokens.fontWeightSemibold,
    marginBottom: "10px",
  },
  helperText: {
    color: tokens.colorNeutralForeground3,
    marginBottom: "10px",
  },
  scrollPane: {
    maxHeight: "300px",
    overflowY: "auto",
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
  },
  emptyState: {
    color: tokens.colorNeutralForeground3,
  },
  citationCard: {
    border: `1px solid ${tokens.colorNeutralStroke1}`,
    borderRadius: "8px",
    padding: "12px",
    marginBottom: "12px",
    backgroundColor: tokens.colorNeutralBackground2,
    cursor: "pointer",
  },
  label: {
    fontWeight: tokens.fontWeightSemibold,
    marginTop: "6px",
    marginBottom: "4px",
  },
  text: {
    wordBreak: "break-word",
  },
});

const CitationList: React.FC<CitationListProps> = (props: CitationListProps) => {
  const styles = useStyles();

  const handleCitationClick = async (commentId: string) => {
    try {
      const found = await selectCommentById(commentId);

      if (!found) {
        console.log("Could not find the linked citation comment in the document.");
      }
    } catch (error) {
      console.error("Error jumping to citation location:", error);
    }
  };

  return (
    <div className={styles.container}>
      <div className={styles.title}>References</div>
      <div className={styles.helperText}>Click a reference to jump to its in-text citation.</div>

      <div className={styles.scrollPane}>
        {props.citations.length === 0 ? (
          <div className={styles.emptyState}>No references yet.</div>
        ) : (
```

```
          props.citations.map((citation) => (
            <div
              key={citation.id}
              className={styles.citationCard}
              onClick={() => handleCitationClick(citation.commentId)}
            >
              <div className={styles.label}>Citation</div>
              <div className={styles.text}>{citation.citationText}</div>

              <div className={styles.label}>Source ID</div>
              <div className={styles.text}>{citation.sourceId}</div>

              <div className={styles.label}>Selected Text</div>
              <div className={styles.text}>{citation.selectedText}</div>
            </div>
          ))
        )}
      </div>
    </div>
  );
};

export default CitationList;
```

Make sure your type includes `commentId`

```
export interface CitationItem {
  id: string;
  selectedText: string;
  sourceId: string;
  citationText: string;
  confidence?: number;
  url?: string;
  commentId: string;
}
```

Why this is better than text search

Text search is okay for MVP, but it can jump to the wrong place if the same text appears multiple times. A comment-based jump is better because the comment is attached to the exact citation range you inserted, and the API lets you retrieve that exact range later. Microsoft Learn+1

One practical note

Use **`context.document.body.getComments()`** for this approach. The body comment collection is documented for WordApi 1.4 and is the safer fit for your Word-on-the-web target. Microsoft Learn+1

If you want to jump to the highlighted source text instead

That is harder with your current setup unless you also store a separate anchor for the original selected range. Since your inserted comment is attached to the in-text citation you created, the cleanest version right now is:

- click reference
- jump to the in-text citation comment

That gives you a reliable, document-native anchor.

What to test

Try this flow:

1. select text
2. analyze
3. confirm highlight + inserted in-text citation + comment
4. confirm a reference appears in the pane
5. click the reference
6. verify Word jumps to the in-text citation location

If you want, the next refinement is to make the reference card briefly highlight or focus visually after click so the interaction feels more polished.

> Can you export this chat as a pdf?

I can't directly export the entire conversation thread from the ChatGPT interface to a PDF because I don't have access to the full chat transcript outside what's visible in messages. But you have a few easy options to generate a clean PDF.

Option 1 (Fastest – Built-in Browser Print)

1. Scroll to the **top of the conversation** so everything loads.
2. Press **Ctrl + P** (Windows) or **Cmd + P** (Mac).
3. In the printer dropdown choose **"Save as PDF."**
4. Set:
    - **Layout:** Portrait
    - **Pages:** All
    - **Background graphics:** On (optional)
5. Click **Save**.

This captures the entire chat exactly as shown.

Option 2 (Cleaner Formatting)

If you want a nicer looking document:

1. Click inside the chat.
2. Press **Ctrl + A** to select everything.
3. **Copy**.
4. Paste into **Google Docs / Word / Notion**.

5. Export as **PDF**.

This lets you:

- remove DevTools screenshots
- add headings
- organize code blocks.

Option 3 (Best for your interview documentation)

If you'd like, I can also generate a **structured PDF-style document** from this chat that includes:

- Project architecture overview
- Code snippets you implemented
- Step-by-step development log
- Final feature list
- Future improvements

It would look like a **technical implementation report** you could even attach to your interview submission.

If you want that, tell me and I'll format it like:

```
Citation Assistant — Word Add-in
Implementation Summary

1. API Integration
2. Selection Detection
3. Citation Parsing
4. In-text Citation Injection
5. Comment Anchoring
6. Reference Pane
7. Click-to-Jump Navigation
```

and make it **cleanly exportable to PDF**.